

Statistical Methods for single cell data analysis (focusing on technology)

Yongjin Park, UBC Path + Stat, BC Cancer

10 March 2025

Next Lectures – single cell genomics

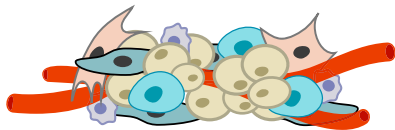
- Lecture 16: technology and data normalization
- Lecture 17: unsupervised learning -1
- Lecture 18: unsupervised learning -2

Today's lecture

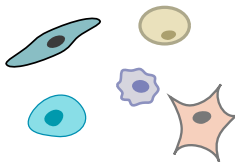
- 1 Single-cell sequencing technology
- 2 Basic quality control
- 3 Additional Q/C tools
- 4 Doublet detection in single-cell data
- 5 Data normalization across many batches

Droplet-based single-cell sequencing technology

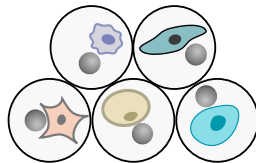
tissue sample



a mixture of cells

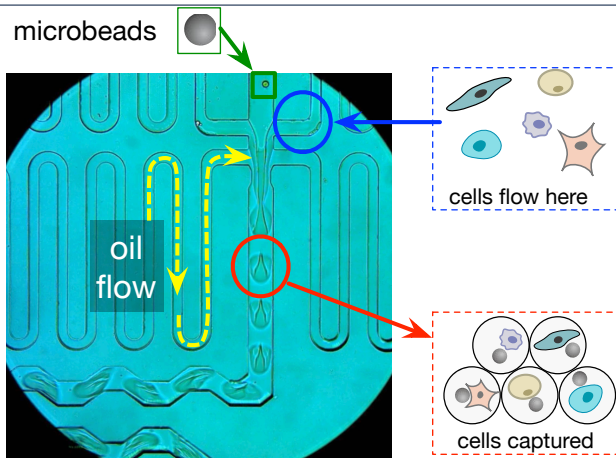


one drop = one cell



Macosko *et al.*, *Cell* (2015)

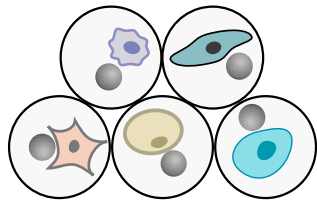
Drop-seq idea 1: Capture one cell with a microbead in a droplet



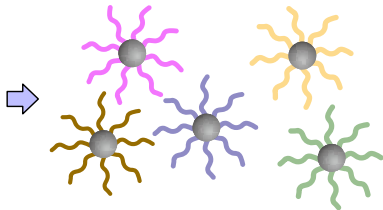
Macosko *et al.*, *Cell* (2015)

Drop-seq idea 2: Massively-parallel sequencing followed by cell-specific barcoding

one drop = one cell



all the genes attached to the microbeads

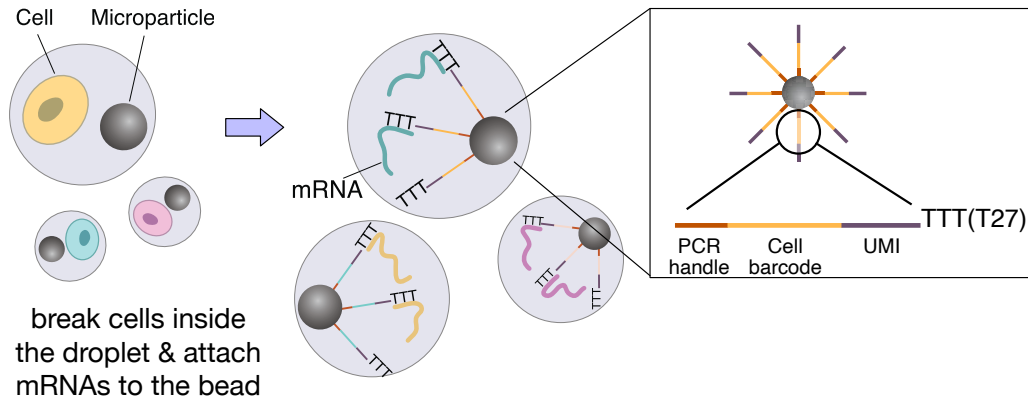


Massively parallel sequencing by mixing them all



Macosko *et al.*, *Cell* (2015)

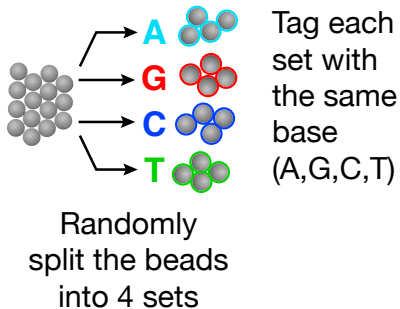
Drop-seq idea 3: How do we keep track of mRNA short reads' membership to a certain droplet?



Macosko *et al.*, *Cell* (2015)

How do we construct millions of unique barcodes?

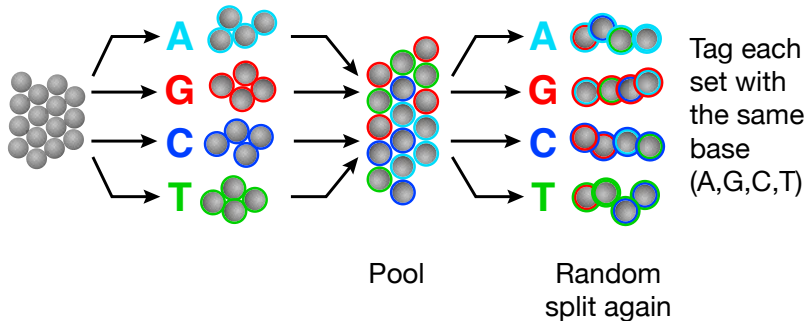
Use DNA as a hashing function!



Macosko *et al.*, *Cell* (2015)

How do we construct millions of unique barcodes?

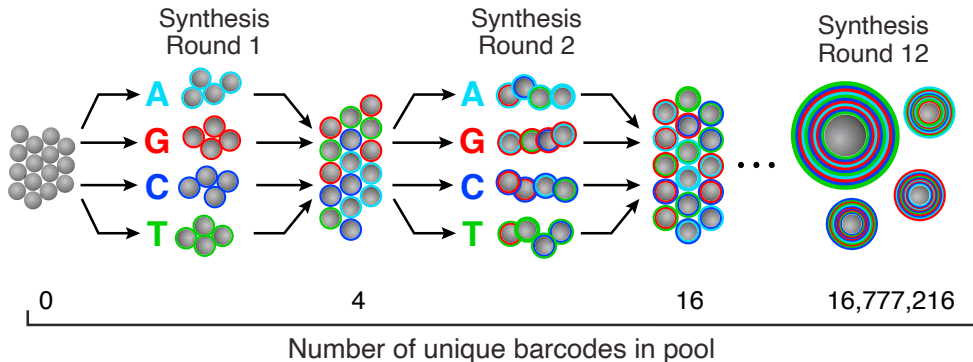
Use DNA as a hashing function!



Macosko *et al.*, *Cell* (2015)

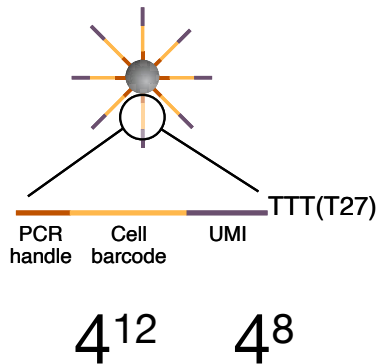
How do we construct millions of unique barcodes?

Use DNA as a hashing function!



Macosko *et al.*, *Cell* (2015)

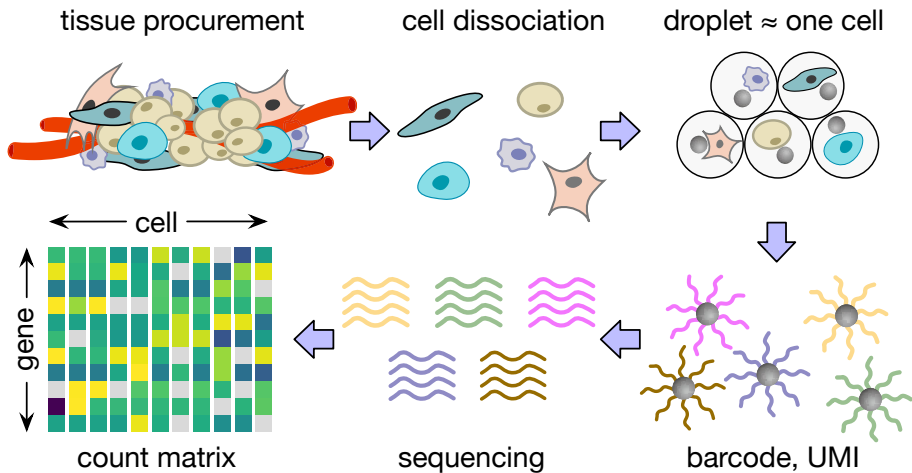
The lengths of barcode sequences determine data dimensionality



- Millions of the same **cell barcode** per bead
- 4^8 different **molecular barcodes** (UMIs) per bead

Technically, we can build up to a $65,536 \times 16,777,216$, gene $\times$ cell expression matrix in one single-cell RNA-seq experiment.

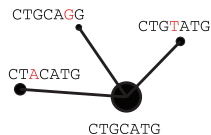
Droplet-based single-cell sequencing pipeline



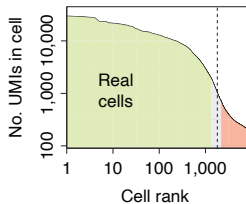
Macosko *et al.*, *Cell* (2015)

Overview of single-cell data analysis

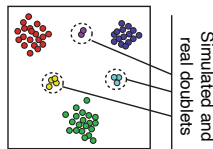
Alignment and molecular counting



Cell filtering and quality control



Doublet scoring

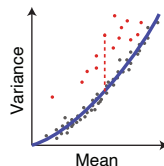


Cell size estimation

Cells	c_1	c_2	c_3
Gene ₁	2	4	20
Gene ₂	1	2	10
Gene ₃	3	6	30

Cell depth: 6 | 12 | 60

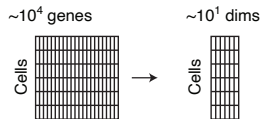
Gene variance analysis



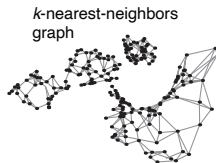
Karchenko, *Nature Methods* (2021)

Overview of single-cell data analysis cont'd

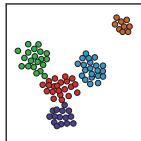
Reduction to a medium-dimensional space



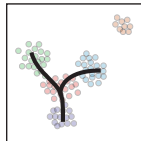
Manifold representation



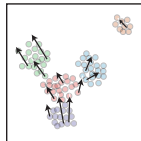
Clustering and differential expression



Trajectories



Velocity estimation

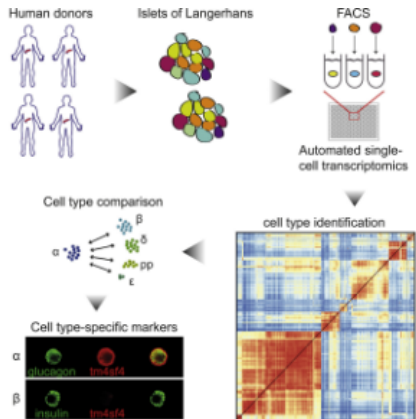


Karchenko, *Nature Methods* (2021)

Today's lecture

- 1 Single-cell sequencing technology
- 2 Basic quality control**
- 3 Additional Q/C tools
- 4 Doublet detection in single-cell data
- 5 Data normalization across many batches

Example: human pancreatic cells (Muraro *et al.* 2016)



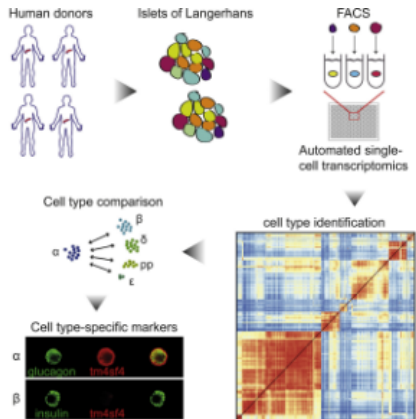
It's a sparse matrix...

```
X[1:10, 1:5]
```

```
## 10 x 5 sparse Matrix of class "dgCMatrix"
##           D28-1_1 D28-1_2 D28-1_3 D28-1_4 D28-1_5
## A1BG-AS1 .         .         .         .         .
## A1BG      .         .         1.00196 1.00196      .
## A1CF      6.07143 .         2.00785 6.07143      .
## A2M-AS1   .         .         .         .         .
## A2ML1     .         .         .         .         .
## A2M       .         5.04947 .         .         .
## A4GALT    .         1.00196 .         .         .
## A4GNT     .         .         .         .         .
## AAAS      1.00196 .         .         1.00196      .
## AACSP1    .         .         .         .         .
```

- What are the rows and columns?

Example: human pancreatic cells (Muraro *et al.* 2016)



It's a sparse matrix...

```
X[1:10, 1:5]
```

```
## 10 x 5 sparse Matrix of class "dgCMatix"
##           D28-1_1 D28-1_2 D28-1_3 D28-1_4 D28-1_5
## A1BG-AS1 .         .         .         .         .
## A1BG      .         .         1.00196 1.00196      .
## A1CF      6.07143 .         2.00785 6.07143      .
## A2M-AS1   .         .         .         .         .
## A2ML1     .         .         .         .         .
## A2M       .         5.04947 .         .         .
## A4GALT    .         1.00196 .         .         .
## A4GNT     .         .         .         .         .
## AAAS      1.00196 .         .         1.00196      .
## AACSP1    .         .         .         .         .
```

- rows: 19,140 genes
- columns: 3,072 cells

Computing row-wise (genes) statistics

- Statistics across cells (columns) per gene (row)

```
head(row.scores)[, .(gene, nnz, mean, sd, cv, sum, sum.sq)]
```

##	gene	nnz	mean	sd	cv	sum	sum.sq
## 1:	A1BG-AS1	3	0.0009784766	0.03130105	31.989576	3.00588	3.011772
## 2:	A1BG	256	0.0975982472	0.35768881	3.664910	299.82181	422.169739
## 3:	A1CF	1525	1.8421998024	3.05790520	1.659920	5659.23779	39141.703125
## 4:	A2M-AS1	60	0.0208792929	0.15205784	7.282710	64.14119	72.345612
## 5:	A2ML1	53	0.0176138654	0.13416946	7.617264	54.10979	56.235519
## 6:	A2M	148	0.2410295606	2.15662813	8.947567	740.44281	14461.827148

- nnz: number of non-zero elements
- mean: average; sd: standard deviation; cv: coefficient of variation
- sum: sum of all the counts; sum.sq: sum of all the squared counts

Computing column-wise (cells) statistics

- Statistics across genes (rows) per cell (column)

```
head(col.scores)
```

##	name	nnz	mean	sd	cv	sum	sum.sq
## 1:	D28-1_1	5448	1.22457922	19.0589027	15.563634	23438.4453	6980786.50
## 2:	D28-1_2	6464	1.67512119	9.8313141	5.869017	32061.8203	1903582.62
## 3:	D28-1_3	5212	0.90166962	12.9791527	14.394577	17257.9570	3239686.75
## 4:	D28-1_4	7318	1.78534663	10.8671150	6.086838	34171.5352	2321212.75
## 5:	D28-1_5	4666	0.84562463	5.1858549	6.132573	16185.2559	528393.56
## 6:	D28-1_6	210	0.02899498	0.9003856	31.053156	554.9639	15531.96

- nnz: number of non-zero elements
- mean: average; sd: standard deviation; cv: coefficient of variation
- sum: sum of all the counts; sum.sq: sum of all the squared counts

QC #1. Column filtering to ensure the quality of cells

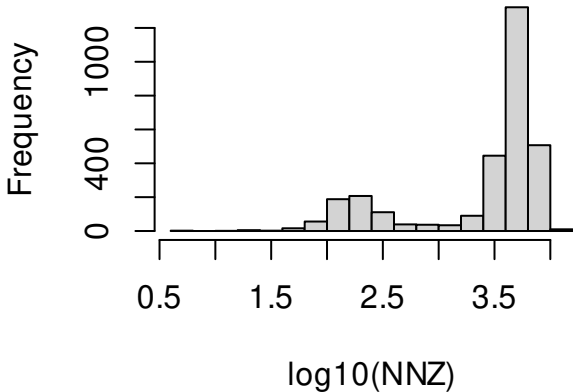
Why cell Q/C?

- Too **few** genes expressed:
- Unusual **high** expressions:

QC #1. Column filtering to ensure the quality of cells

Why cell Q/C?

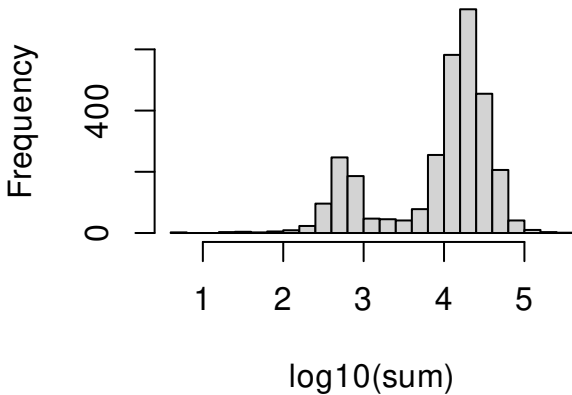
- Too **few** genes expressed: dying, **ambient cells**, too many technical drop out
- Unusual **high** expressions:



QC #1. Column filtering to ensure the quality of cells

Why cell Q/C?

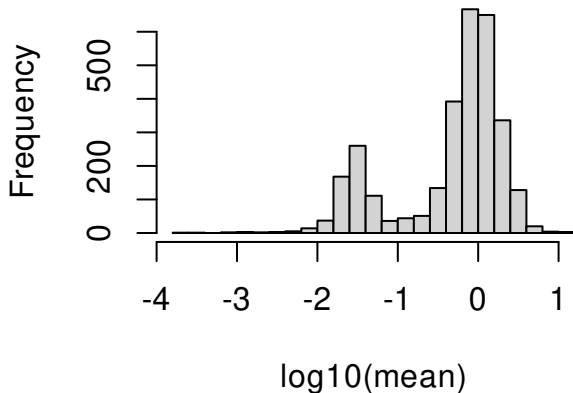
- Too **few** genes expressed: dying, **ambient cells**, too many technical drop out
- Unusual **high** expressions: **doublers**, high ribosomal genes (background activities), disequilibrium



QC #1. Column filtering to ensure the quality of cells

Why cell Q/C?

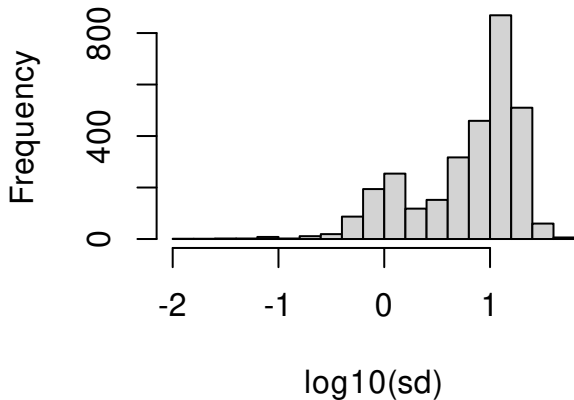
- Too **few** genes expressed: dying, **ambient cells**, too many technical drop out
- Unusual **high** expressions: **doublers**, high ribosomal genes (background activities), disequilibrium



QC #1. Column filtering to ensure the quality of cells

Why cell Q/C?

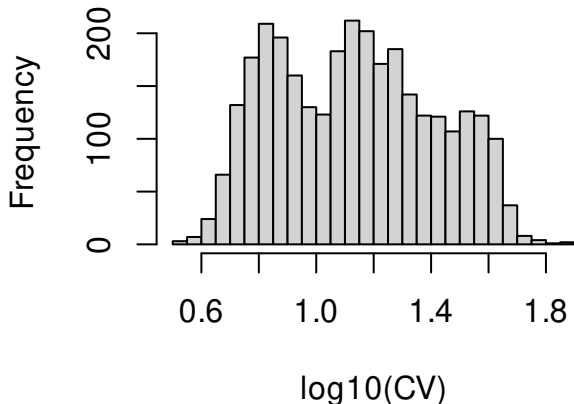
- Too **few** genes expressed: dying, **ambient cells**, too many technical drop out
- Unusual **high** expressions: **doublers**, high ribosomal genes (background activities), disequilibrium



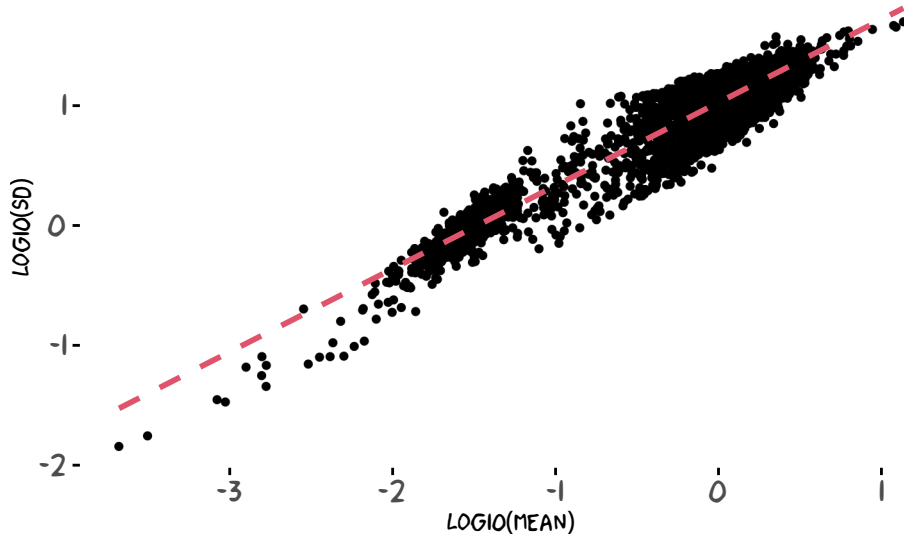
QC #1. Column filtering to ensure the quality of cells

Why cell Q/C?

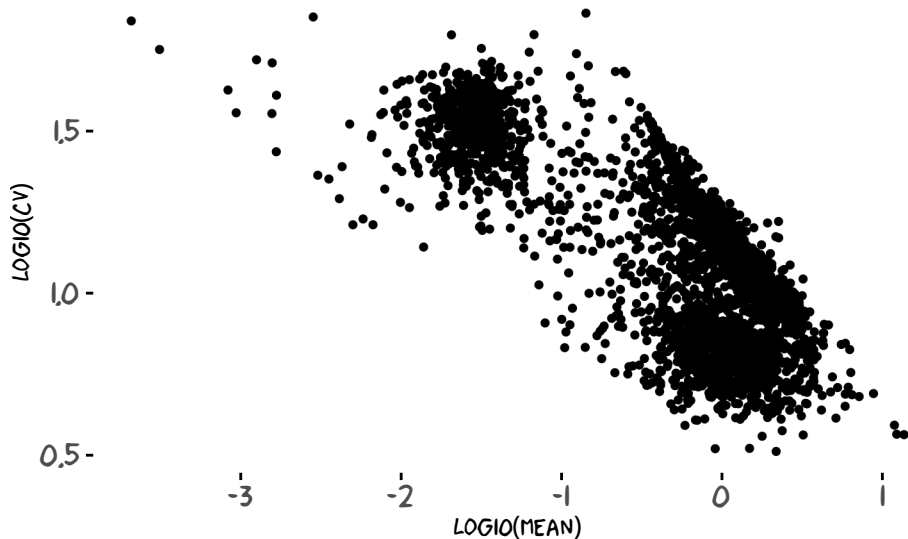
- Too **few** genes expressed: dying, **ambient cells**, too many technical drop out
- Unusual **high** expressions: **doublers**, high ribosomal genes (background activities), disequilibrium



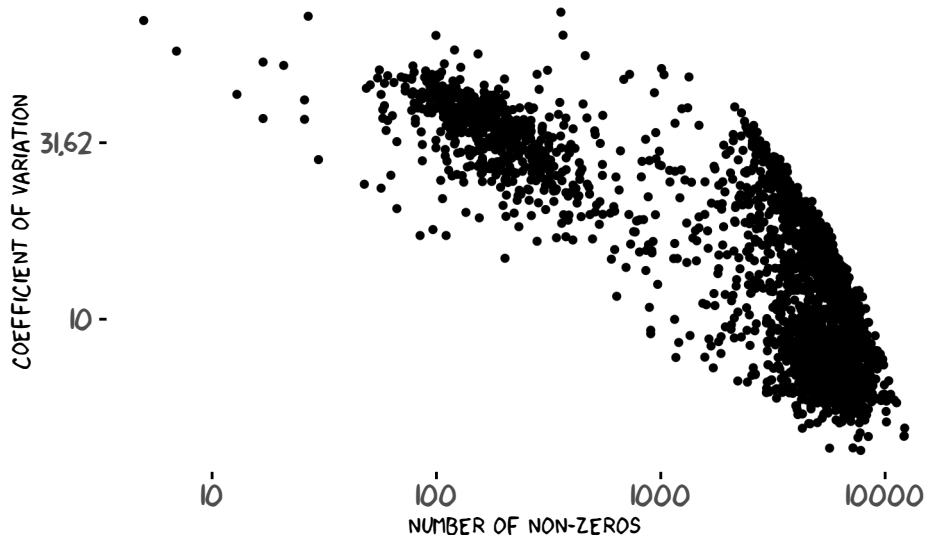
Mean-variance relationship of cell statistics



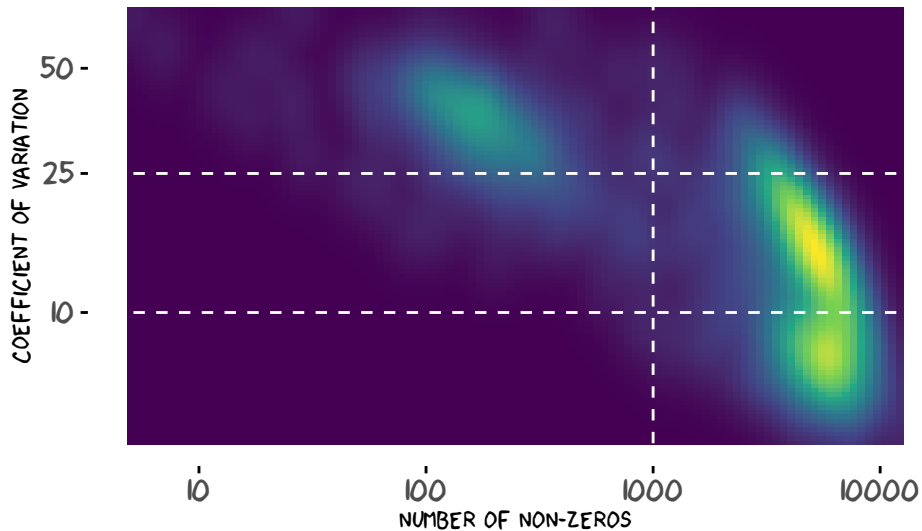
Mean-variance relationship of cell statistics



Robust Q/C by putting multiple scores together



Robust Q/C by putting multiple scores together



Can we systematically dissect “axes of variation?”

Unsupervised Learning Methods considered in single-cell analysis:

① **Principal** Component Analysis (PCA)

Can we systematically dissect “axes of variation?”

Unsupervised Learning Methods considered in single-cell analysis:

- ① **Principal** Component Analysis (PCA)
- ② (more general) matrix factorization (lect 17)
- ③ (Variational) autoencoder (lect 18)

Can we systematically dissect “axes of variation?”

Unsupervised Learning Methods considered in single-cell analysis:

- ① **Principal** Component Analysis (PCA)
- ② (more general) matrix factorization (lect 17)
- ③ (Variational) autoencoder (lect 18)
- ④ GPT (Generative Pretrained Transformer; *ask ChatGPT*)

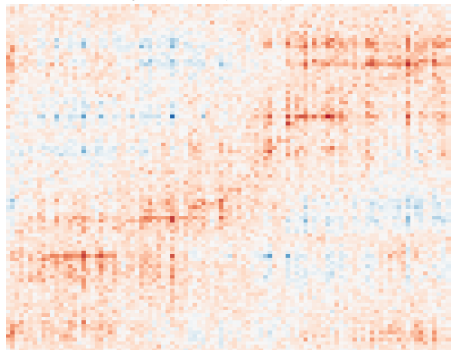
What is Principal Component Analysis?

The goal is to find hidden patterns

A data matrix was generated by the following:

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \epsilon_i$$

for all i with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I)$



Can we reverse engineer the U and V matrices from the data X ?

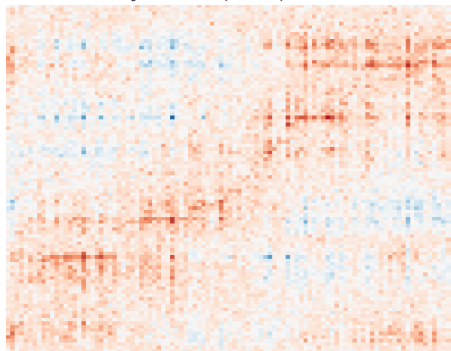
$$X \rightarrow UV^\top$$

The goal is to find hidden patterns

A data matrix was generated by the following:

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \epsilon_i$$

for all i with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I)$



Can we reverse engineer the U and V matrices from the data X ?

$$X \rightarrow UV^\top$$

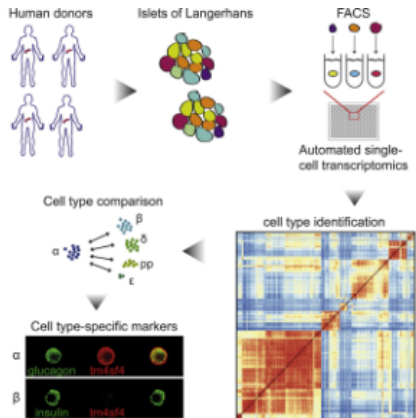
FEATURES



FEATURE LOADING

X

Example: Pancreatic cells (Muraro *et al.* 2016)



It's a sparse matrix...

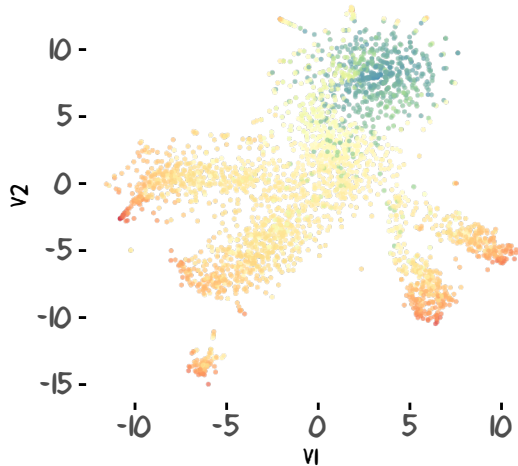
```
x[1:10, 1:5]
```

```
## 10 x 5 sparse Matrix of class "dgCMatrix"
##           D28-1_1 D28-1_2 D28-1_3 D28-1_4 D28-1_5
## A1BG-AS1      .      .      .      .      .
## A1BG          .      .      1      1      .
## A1CF          6      .      2      6      .
## A2M-AS1       .      .      .      .      .
## A2ML1         .      .      .      .      .
## A2M           .      5      .      .      .
## A4GALT        .      1      .      .      .
## A4GNT         .      .      .      .      .
## AAAS          1      .      .      1      .
## AACSP1        .      .      .      .      .
```

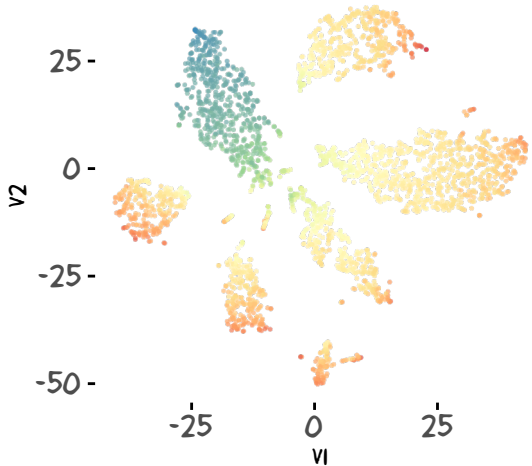
- rows: 19,049 genes
- columns: 3,072 cells

Visualization with or without dimensionality reduction

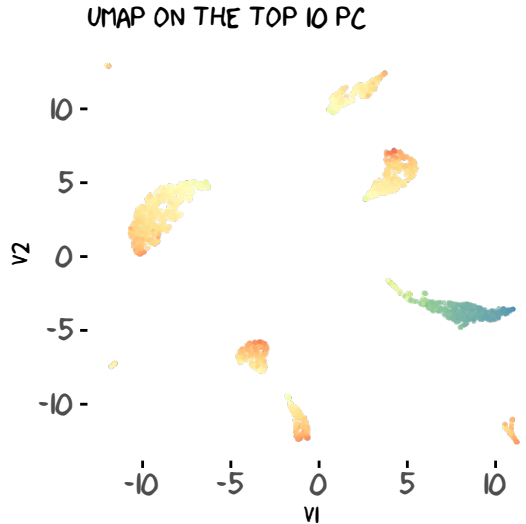
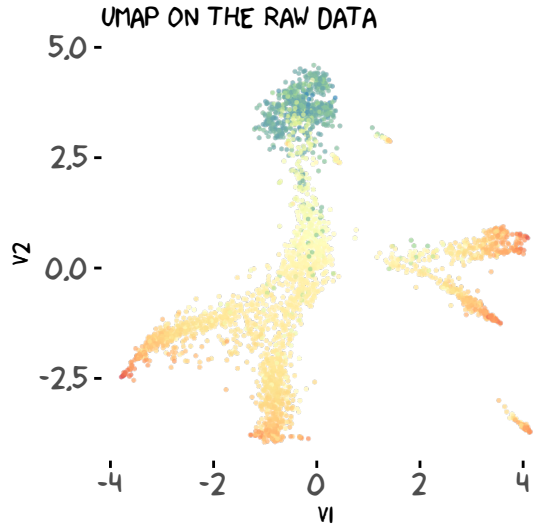
TSNE ON THE RAW DATA



TSNE ON THE TOP 10 PC

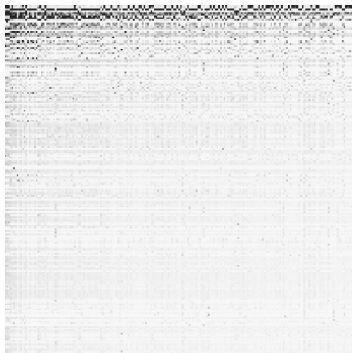


Visualization with or without dimensionality reduction



Let's just use top 200 most variable genes and cells

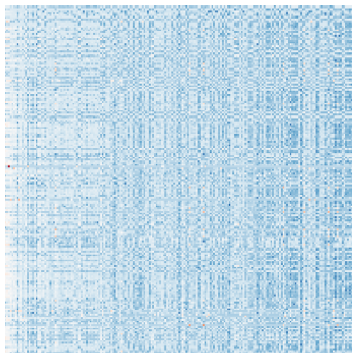
Top 200 genes and cells



- We will call such a high-dimensional matrix X ($m \times n$)
- X has $m=19,049$ rows (transcripts/genes/features)
- X has $n=3,072$ columns (cells/#data points)
- The rows were log-transformed and scaled by `scale()` for visualization
- Each cell is a 19,049-dimensional vector!
- Each gene is a 3,072-dimensional vector...

Let's just use top 200 most variable genes and cells

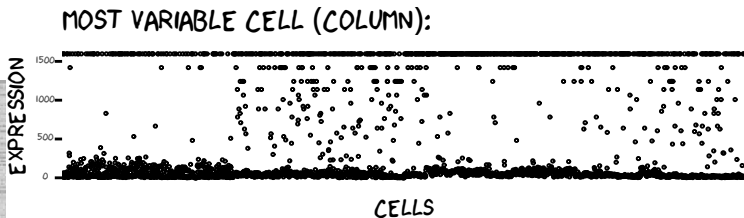
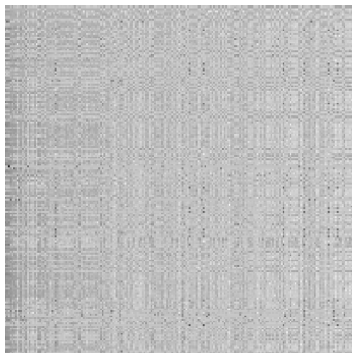
Top 200 genes and cells



- We will call such a high-dimensional matrix X ($m \times n$)
- X has $m=19,049$ rows (transcripts/genes/features)
- X has $n=3,072$ columns (cells/#data points)
- The rows were log-transformed and scaled by `scale()` for visualization
- Each cell is a 19,049-dimensional vector!
- Each gene is a 3,072-dimensional vector...

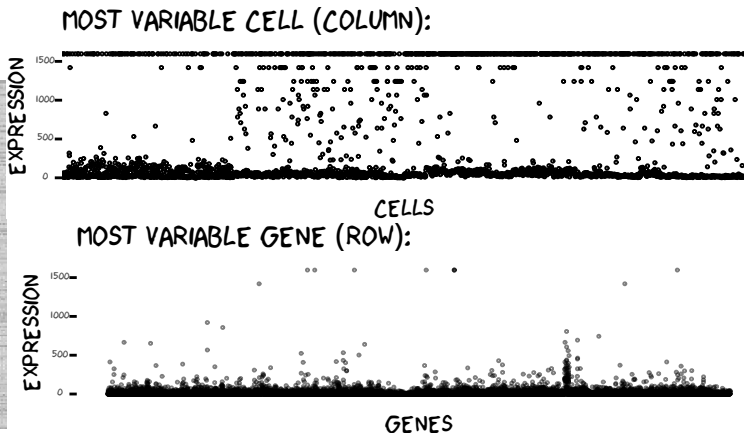
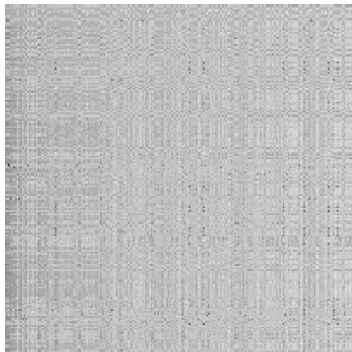
1-dimensional representations/summary of data matrix

Top 200 genes and
cells:



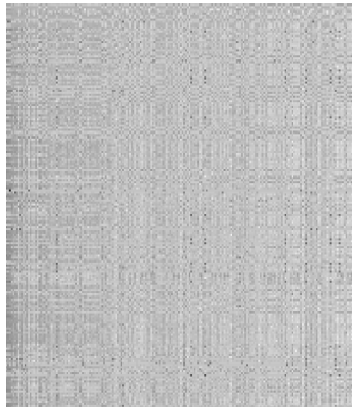
1-dimensional representations/summary of data matrix

Top 200 genes and
cells:



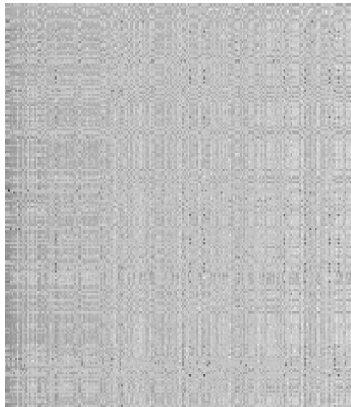
Can you see some common patterns?

200 GENES AND CELLS

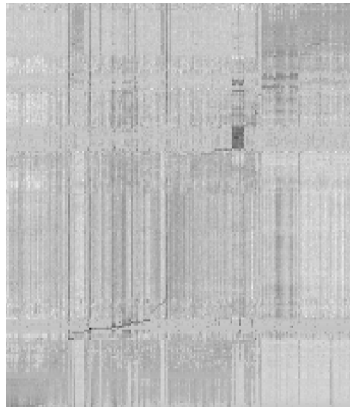


Can you see some common patterns?

200 GENES AND CELLS



REARRANGE THEM...



How do we recover “common” patterns from data?

Principal Component Analysis

(Pearson 1901)

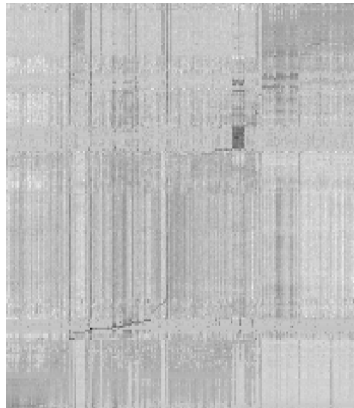
- Projection [of the original data] that minimizes the projection cost between the original and projected
- The cost = mean squared distance

(Hotelling 1933)

- Orthogonal projection of data into a lower-dimensional **[principal]** sub-space,
- such that the total **variation of the projected** is maximized

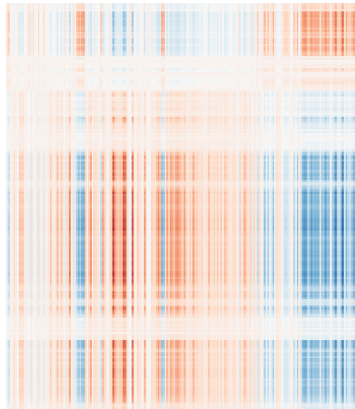
Let's see how much the first eigenvector can explain

TOP 200 GENES AND CELLS



Let's see how much the first eigenvector can explain

EIGEN PROJ.



E-VEC



LOADING



=

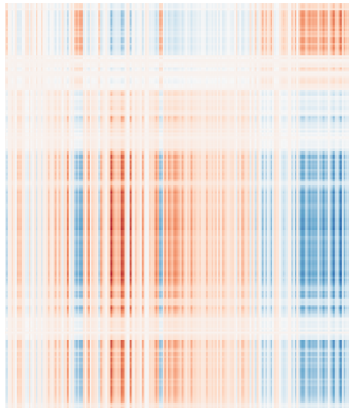
X

Let's see how much the first eigenvector can explain

TOP 200 GENES AND 200 CELL



29%



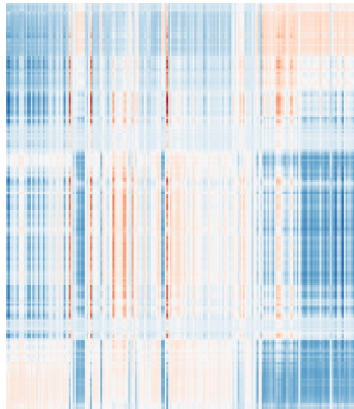
Top two eigen-vectors

TOP 200 GENES AND 200 CELL



Top two eigen-vectors

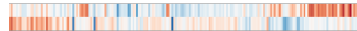
EIGEN PROJ.



E-VEC



LOADING



=

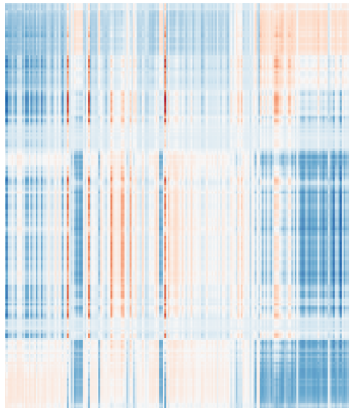
X

Top two eigen-vectors

TOP 200 GENES AND 200 CELL



51%



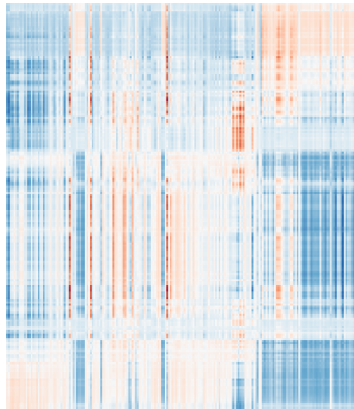
Top three eigen-vectors

TOP 200 GENES AND 200 CELL



Top three eigen-vectors

EIGEN PROJ.



E-VEC



LOADING



=

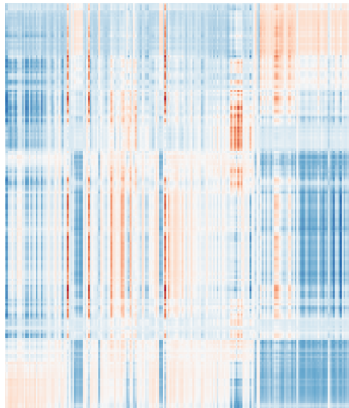
X

Top three eigen-vectors

TOP 200 GENES AND 200 CELL



58%



What is Principal Component Analysis?

PCA: Consider this multivariate linear model

$$\begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{pmatrix} = \begin{pmatrix} U_{11} & \cdots & U_{1k} \\ U_{21} & \cdots & U_{2k} \\ \vdots & \vdots & \vdots \\ U_{p1} & \cdots & U_{pk} \end{pmatrix} \begin{pmatrix} V_1 \\ \vdots \\ V_k \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_p \end{pmatrix}$$

or

$$\mathbf{x} = U\mathbf{v} + \epsilon$$

- If we **knew** U , we would be able to solve weights V .
- And vice versa...

PCA: Consider this multivariate linear model

For many columns, $X = UV + E$, or

$$\begin{pmatrix} X_{11} & \cdots & X_{1n} \\ X_{21} & \cdots & X_{2n} \\ \vdots & & \vdots \\ X_{p1} & \cdots & X_{pn} \end{pmatrix} = \begin{pmatrix} U_{11} & \cdots & U_{1k} \\ U_{21} & \cdots & U_{2k} \\ \vdots & \vdots & \vdots \\ U_{p1} & \cdots & U_{pk} \end{pmatrix} \begin{pmatrix} V_{11} & \cdots & V_{1n} \\ \vdots & & \vdots \\ V_{k1} & \cdots & V_{kn} \end{pmatrix} + \cdots$$

- If we **knew** U , we would be able to solve weights V .
- And vice versa...

PCA: What is a projection matrix?

Suppose we knew V . For each gene (row) g , we solve the following regression.

$$\mathbf{x}_g^\top \sim V\mathbf{u}_g^\top + \epsilon I$$

Least square solution:

$$\min_{\mathbf{u}_g} \|\mathbf{x}_g^\top - V\mathbf{u}_g^\top\|$$

PCA: What is a projection matrix?

Suppose we knew V . For each gene (row) g , we solve the following regression.

$$\mathbf{x}_g^\top \sim V \mathbf{u}_g^\top + \epsilon I$$
$$\begin{pmatrix} X_{g1} \\ \vdots \\ X_{gn} \end{pmatrix} \sim \begin{pmatrix} V_{11} & \cdots & V_{1k} \\ \vdots & \vdots & \vdots \\ V_{n1} & \cdots & V_{nk} \end{pmatrix} \begin{pmatrix} U_{g1} \\ \vdots \\ U_{gk} \end{pmatrix} + \begin{pmatrix} \epsilon \\ \vdots \\ \epsilon \end{pmatrix}$$

Least square solution:

$$\min_{\mathbf{u}_g} \|\mathbf{x}_g^\top - V \mathbf{u}_g^\top\|$$

PCA: What is a projection matrix?

Suppose we knew V . For each gene (row) g , we solve the following regression.

$$\mathbf{x}_g^\top \sim V \mathbf{u}_g^\top + \epsilon I$$
$$\begin{pmatrix} X_{g1} \\ \vdots \\ X_{gn} \end{pmatrix} \sim \begin{pmatrix} V_{11} & \cdots & V_{1k} \\ \vdots & \vdots & \vdots \\ V_{n1} & \cdots & V_{nk} \end{pmatrix} \begin{pmatrix} U_{g1} \\ \vdots \\ U_{gk} \end{pmatrix} + \begin{pmatrix} \epsilon \\ \vdots \\ \epsilon \end{pmatrix}$$

Least square solution:

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\hat{\mathbf{x}}_g^\top = V \hat{\mathbf{u}}_g^\top$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\begin{aligned}\hat{\mathbf{x}}_g^\top &= V \hat{\mathbf{u}}_g^\top \\ &= V(V^\top V)^{-1} V^\top \mathbf{x}_g^\top\end{aligned}$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\begin{aligned}\hat{\mathbf{x}}_g^\top &= V \hat{\mathbf{u}}_g^\top \\ \hat{\mathbf{x}}_g &= \mathbf{x}_g \underbrace{V(V^\top V)^{-1} V^\top}_{n \times n \text{ projection matrix}}\end{aligned}$$

PCA: Projection can be done on the other side

A multivariate regression model for sample (column) j :

$$\mathbf{x}_j \sim U\mathbf{v}_j + \epsilon$$

The least-square solution for the V :

$$\hat{\mathbf{v}}_j = (U^\top U)^{-1} U^\top \mathbf{x}_j$$

Then we have

$$\hat{\mathbf{x}}_j = \underbrace{U(U^\top U)^{-1}U^\top}_{p \times p \text{ projection matrix}} \mathbf{x}_j$$

PCA: maximizing total variance of the projected

Constrained optimization

Given the rank-1 factorization

$$\hat{X} = \mathbf{u}\mathbf{v}^\top$$

and the least square solution $\hat{\mathbf{v}}$

$$\hat{\mathbf{v}}^\top = \frac{\mathbf{u}^\top}{\mathbf{u}^\top \mathbf{u}} X$$

We want to maximize the variance of this projected vector $\mathbf{v}$

$$\hat{\mathbf{v}}^\top \hat{\mathbf{v}} = \frac{\mathbf{u}^\top}{\mathbf{u}^\top \mathbf{u}} X X^\top \frac{\mathbf{u}}{\mathbf{u}^\top \mathbf{u}} = \mathbf{u}^\top X X^\top \mathbf{u}$$

where we assume $\mathbf{u}$ is a unit vector.

PCA is an eigen value problem

PCA

Letting the covariance matrix $\hat{\Sigma} = XX^{\top}/(p-1)$, we want to find a unit vector $\mathbf{v}$ by

$$\max \mathbf{v}^{\top} \hat{\Sigma} \mathbf{v}$$

subject to $\mathbf{v}^{\top} \mathbf{v} = 1$.

Eigen value problem

Given the covariance matrix $\hat{\Sigma}$, we can resolve an eigen-value λ and the corresponding eigen-vector $\mathbf{v}$ such that

$$\hat{\Sigma} \mathbf{v} = \lambda \mathbf{v}$$

SVD: another equivalent method for PCA

Singular Value Decomposition

SVD identifies three matrices of X :

$$X = UDV^\top$$

where both U and V vectors are orthonormal, namely,

- $U^\top U = I$, $\mathbf{u}_k^\top \mathbf{u}_k = 1$ for all k ,
- $V^\top V = I$, $\mathbf{v}_k^\top \mathbf{v}_k = 1$ for all k .

Covariance by SVD

Covariance across the columns (samples)

$$X^\top X / p = VD^2V^\top / p$$

Covariance across the rows (genes)

$$XX^\top / n = UD^2U^\top / n$$

Remark: standardized matrix

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{(X^\top X)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \dots & \dots \\ 0 & D_2^2 & 0 & \dots \\ 0 & \dots & \ddots & 0 \\ 0 & \dots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{(X^T X)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \dots & \dots \\ 0 & D_2^2 & 0 & \dots \\ 0 & \dots & \ddots & 0 \\ 0 & \dots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$
$$\propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \dots & \dots \\ 0 & D_2^2 & 0 & \dots \\ 0 & \dots & \ddots & 0 \\ 0 & \dots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{(X^T X)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \dots & \dots \\ 0 & D_2^2 & 0 & \dots \\ 0 & \dots & \ddots & 0 \\ 0 & \dots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$
$$\propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

SVD: another equivalent method for PCA

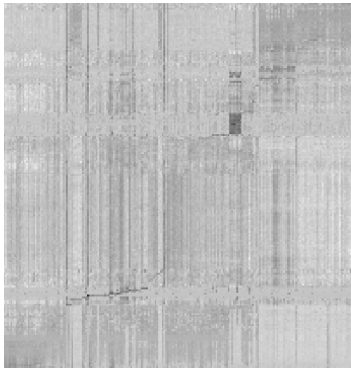
We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{(X^T X)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \dots & \dots \\ 0 & D_2^2 & 0 & \dots \\ 0 & \dots & \ddots & 0 \\ 0 & \dots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$
$$= \underbrace{\frac{D_1^2}{p-1}}_{\text{eigenvalue}} \underbrace{\mathbf{v}_1}_{\text{eigenvector}}$$

Run SVD to find principal components

```
svd.out <- rsvd::rsvd(x.sorted, k = 7)
U <- svd.out$u
D <- diag(svd.out$d)
V <- svd.out$v
```

DATA MATRIX



=

U



X

D



X

TRANSPPOSE(V)



Learning PCA by singular value decomposition

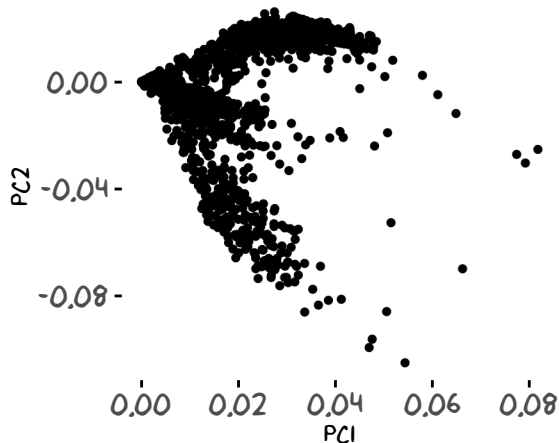
```
library(rsvd)
.svd <- rsvd(X, k = 3)
names(.svd)
```

```
## [1] "d" "u" "v"
```

```
dim(.svd$v)
```

```
## [1] 3072      3
```

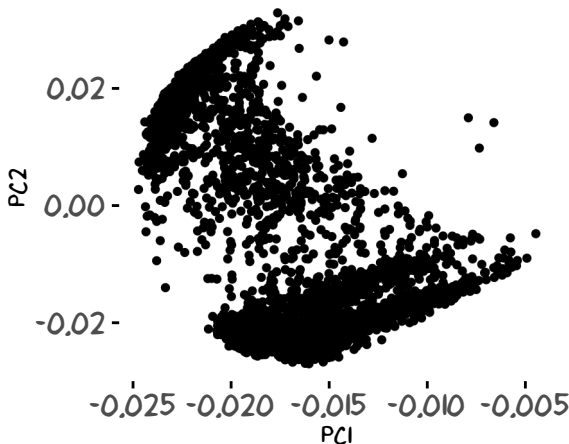
- rsvd provides faster SVD



Learning PCA by singular value decomposition - 2

In single-cell data analysis, it is useful to transform the raw count values to more Gaussian-like ones.

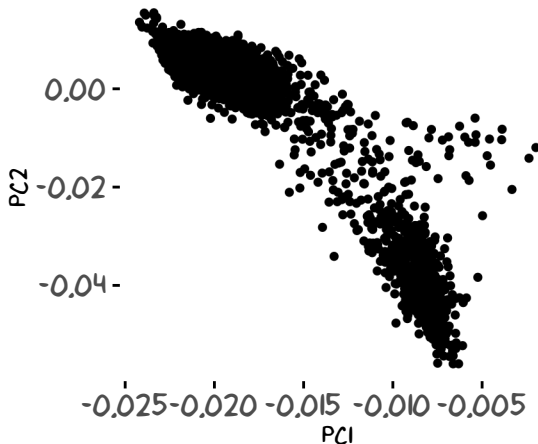
```
X.n <- apply(X, 2, function(x) x/sum(x))  
X.n <- X.n/10000  
log.x <- apply(log1p(X.n), 2, scale)  
.svd <- rsvd(log.x, k = 3)  
names(.svd)  
## [1] "d" "u" "v"  
  
dim(.svd$v)  
## [1] 3072 3
```



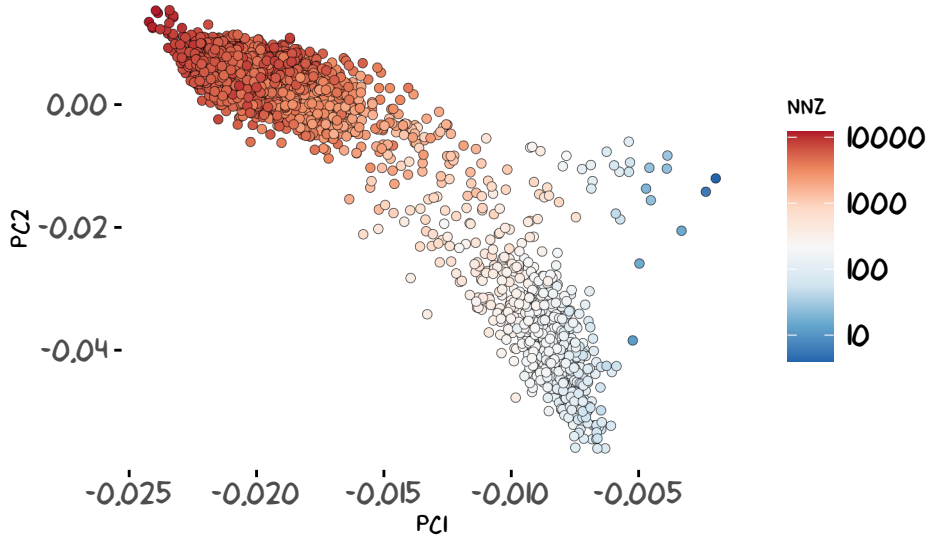
Learning PCA by singular value decomposition - 3

We can also add pseudo-count to make the SVD results smoother..

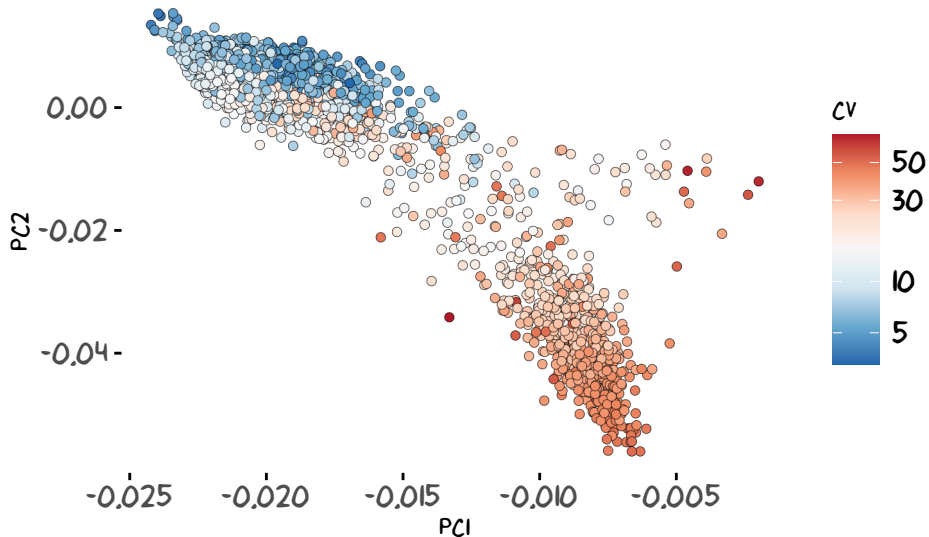
```
names(.svd)
## [1] "U" "D" "V"
dim(.svd$V)
## [1] 3072 3
```



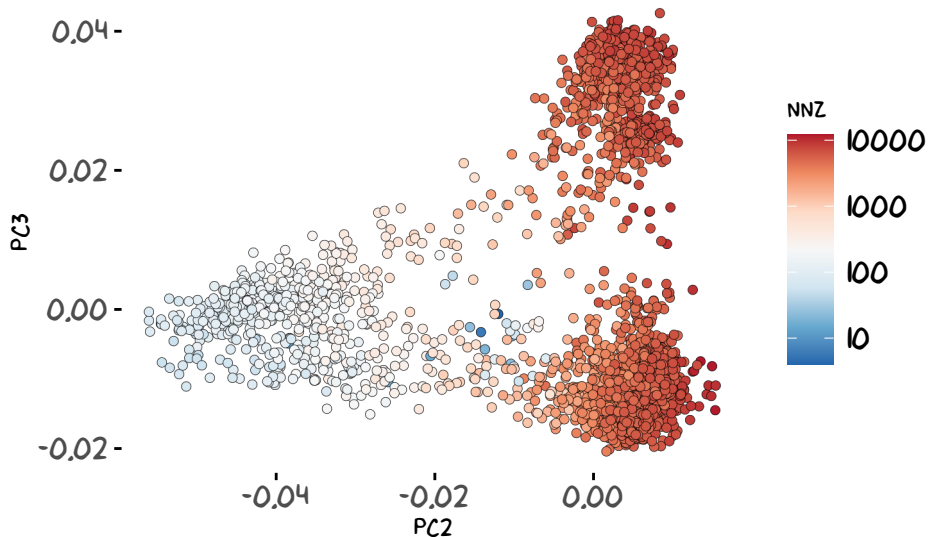
PCA to confirm/remvoe low-quality cells



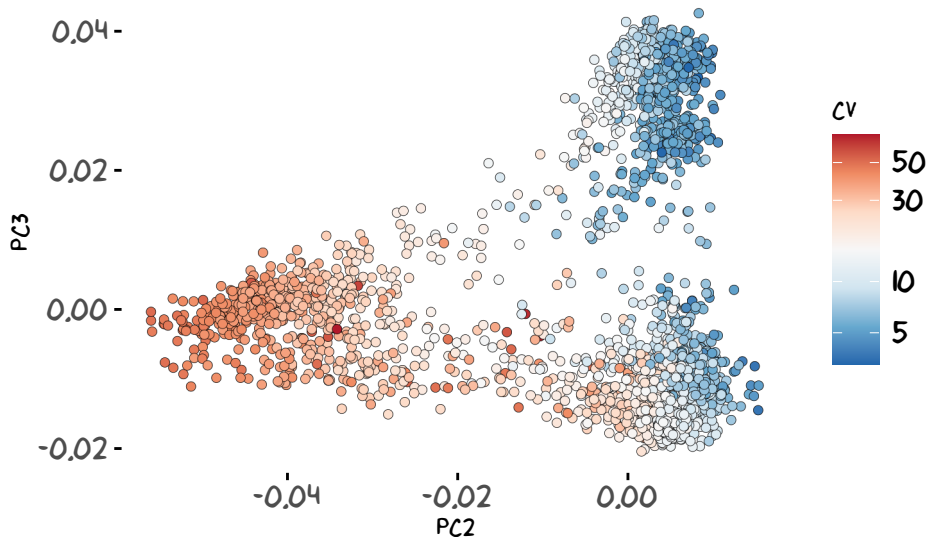
PCA to confirm/remvoe low-quality cells



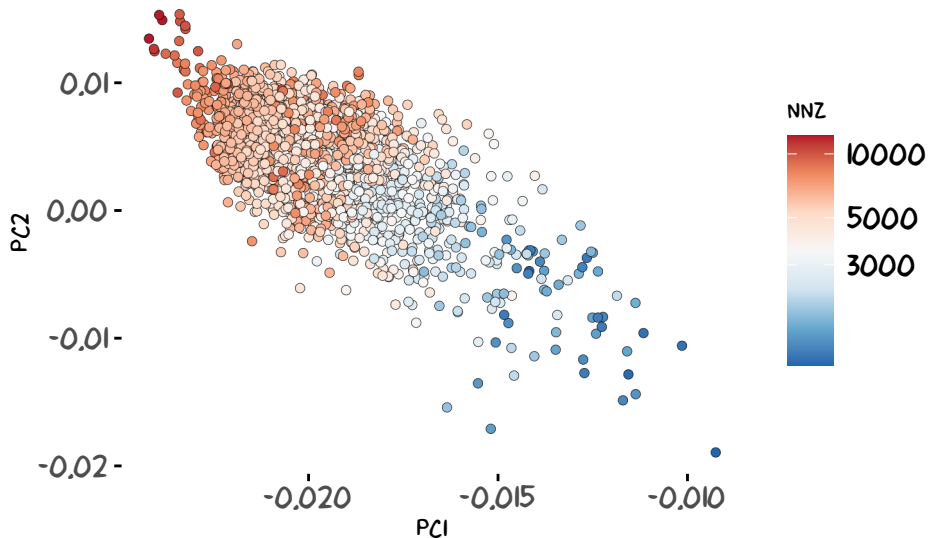
PCA to confirm/remvoe low-quality cells



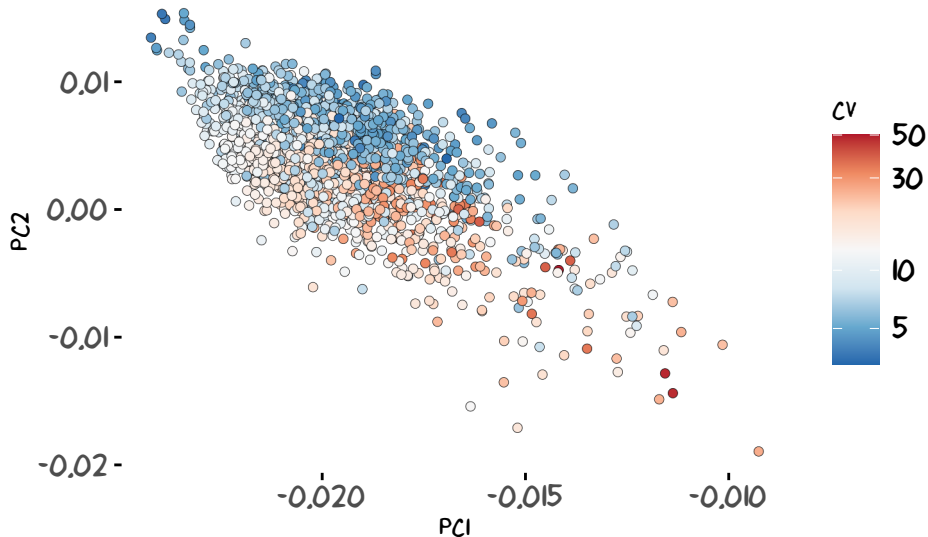
PCA to confirm/remvoe low-quality cells



After removing “dying” cells ...



After removing “dying” cells ...

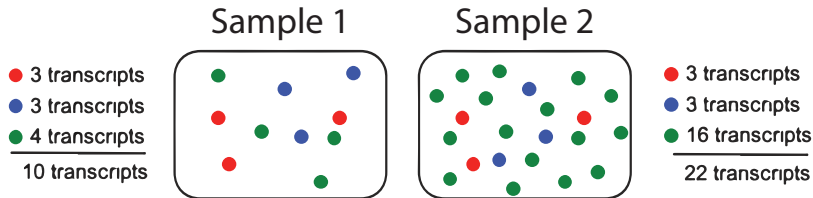


Should we normalize single-cell gene expression vectors?

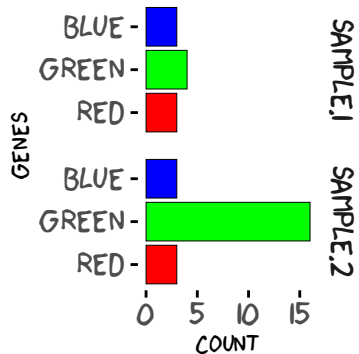
Recall: Should we always normalize within each sample?

What could go wrong with that?

Consider the following example:

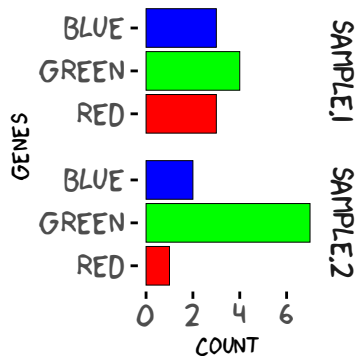


What if we sequence “deep” enough?



##	sample.1	sample.2
## Red	3	3
## Green	4	16
## Blue	3	3

What if we sequence *less* than the actual counts?

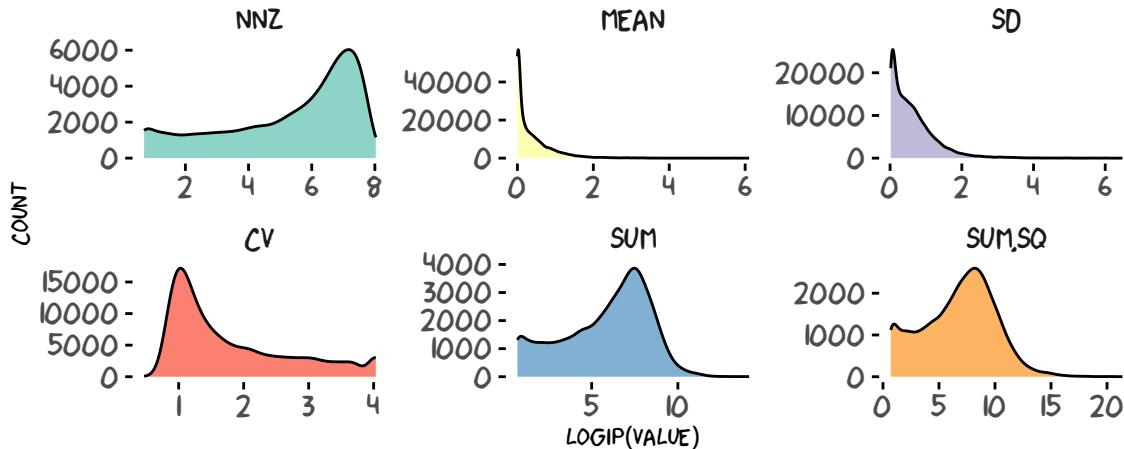


##	sample.1	sample.2
## Red	3	1
## Green	4	7
## Blue	3	2

Question

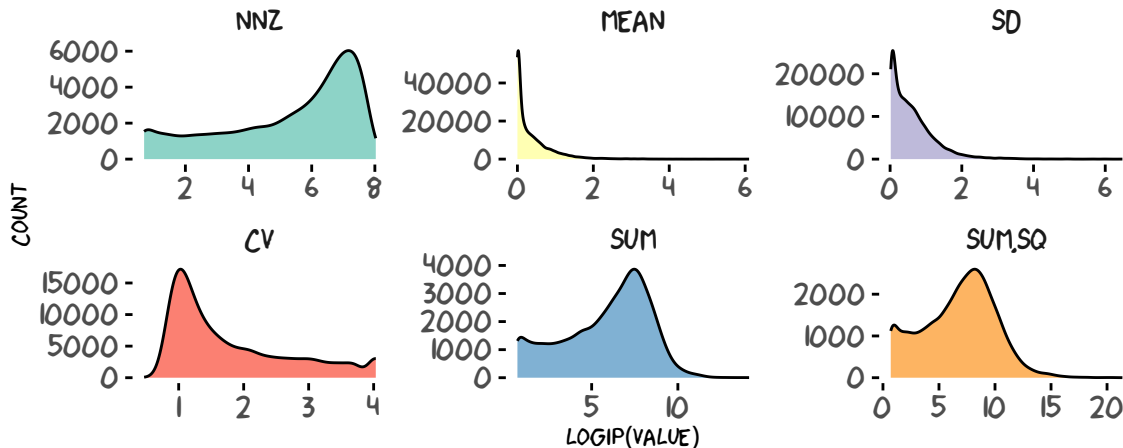
- Is it okay to normalize to have both samples have the same sequence depth?
- Are we comparing the gene “red” with the gene “green” or vice versa?

QC 2. Can we select informative rows/genes?



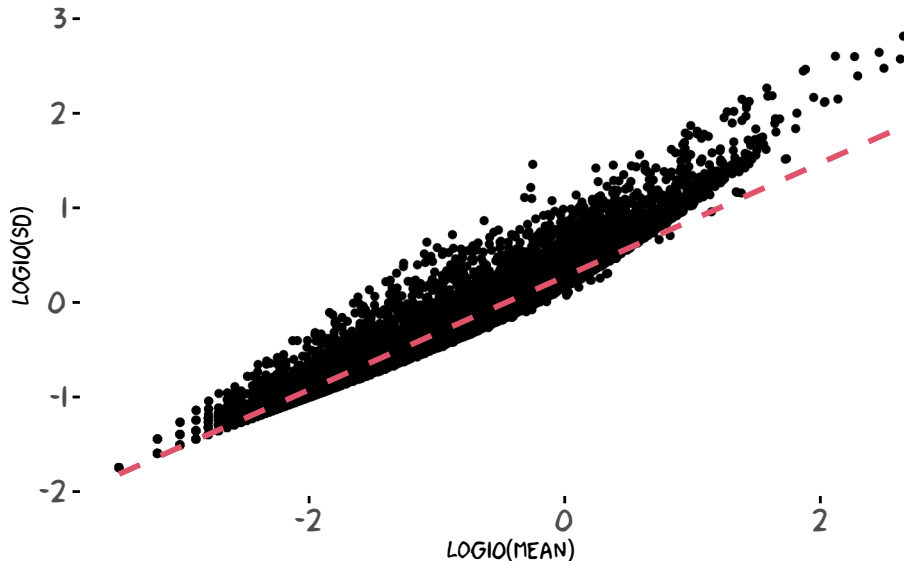
'nnz': number of non-zero elements, 'sd': standard deviation, 'cv': coefficient of variation ('sd'/'mean'), 'sum.sq': sum of squares.

QC 2. Can we select informative rows/genes?

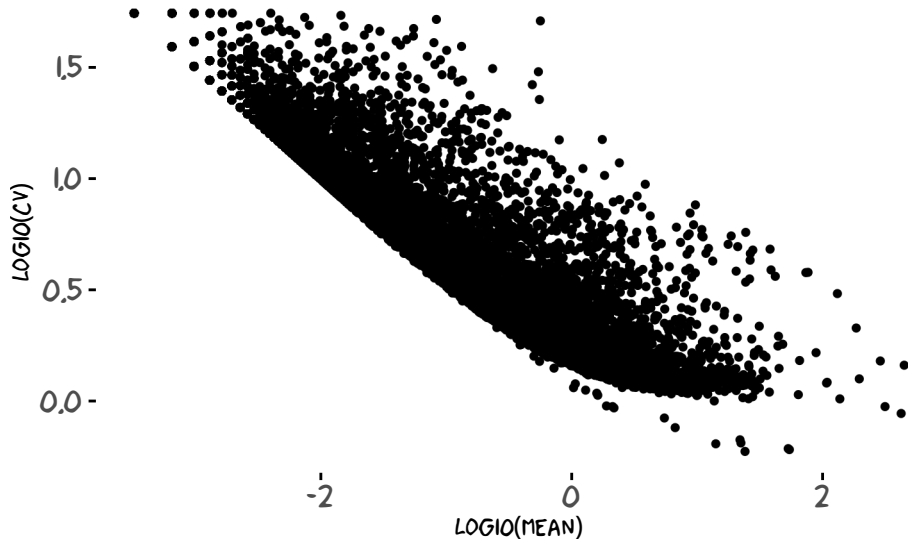


We may select only highly variable genes for the subsequent analysis

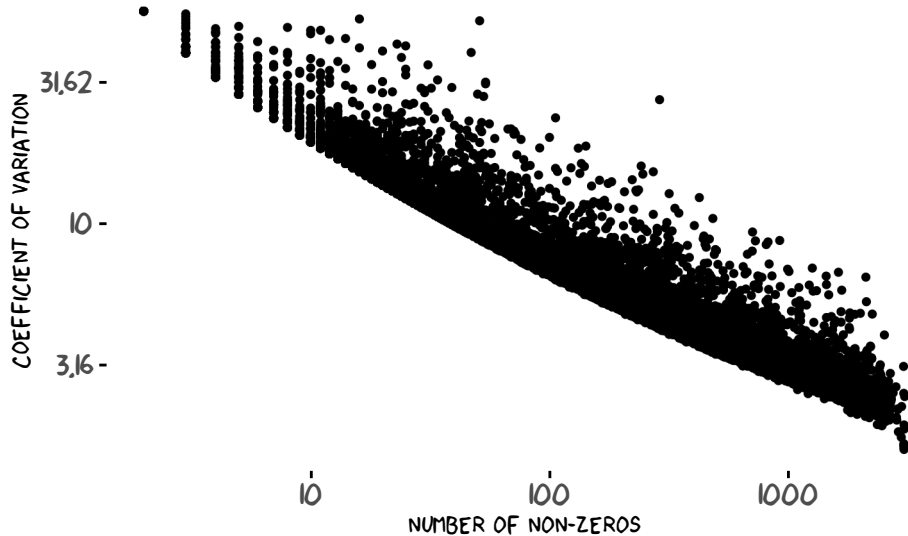
Mean-variance relationship of genes



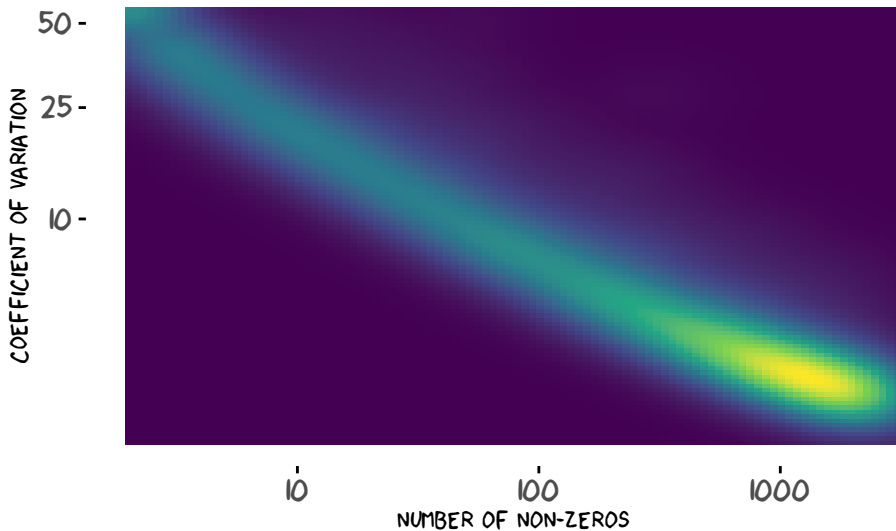
Mean-variance relationship of genes



Gene Q/C: Any obvious cutoff?



Gene Q/C: Any obvious cutoff?



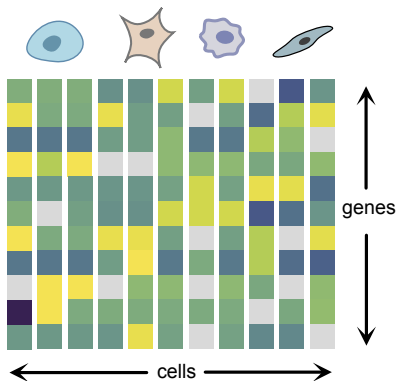
Can we use the count data as they are?

- In bulk RNA-seq, we transformed the count data to adjust size factors that vary across samples.
- In single-cell data analysis, we generally care a lot about zero values, not knowing whether they come from actual “no expression” or technical drop-out.
- Should we take some sort of voom transformation?

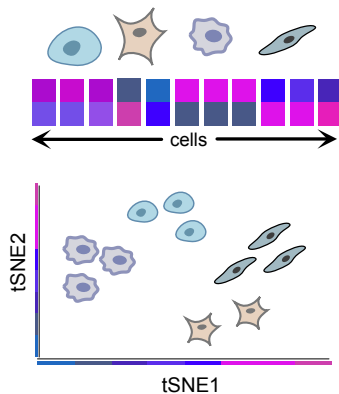
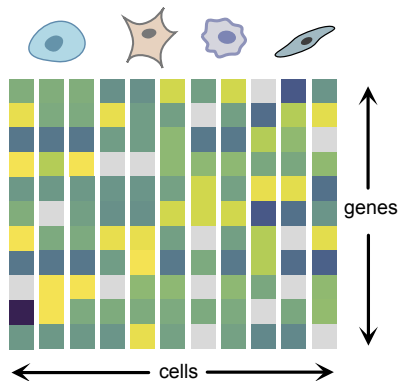
Today's lecture

- 1 Single-cell sequencing technology
- 2 Basic quality control
- 3 Additional Q/C tools**
- 4 Doublet detection in single-cell data
- 5 Data normalization across many batches

It's useful to show high-dimensional data in 2D

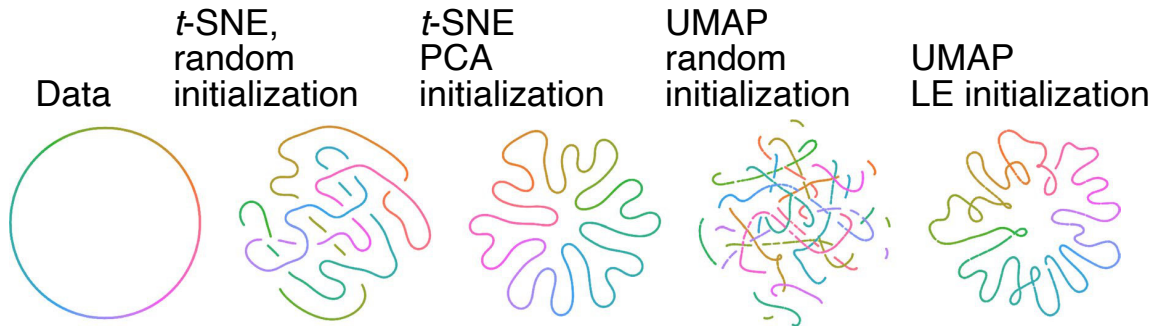


It's useful to show high-dimensional data in 2D



tSNE: t-distributed Stochastic Neighbourhood Embedding (Van der Maaten & Hinton, 2008).

Warning: Don't over interpret 2D embedding



Kobak and Berens, *Nature Biotech* (2021)

Don't over interpret 2D embedding results

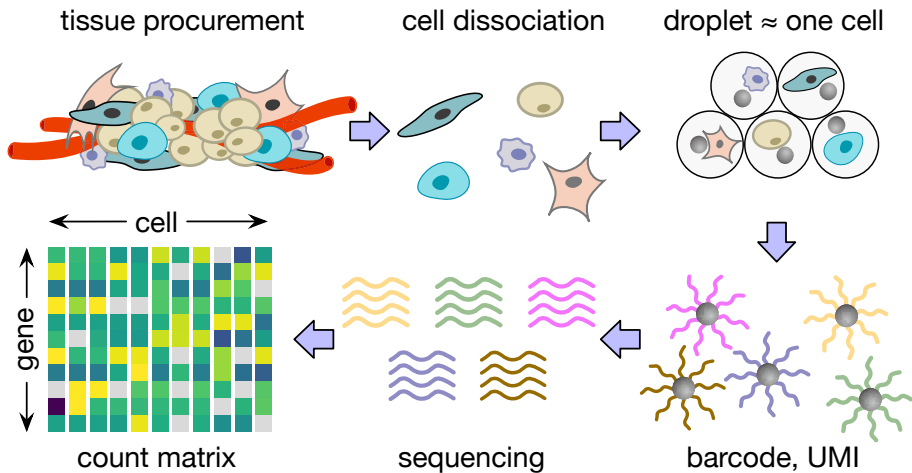
Check out these papers:

- Initialization is critical for preserving global data structure in both t-SNE and UMAP
- The art of using t-SNE for single-cell transcriptomics
- Dimensionality reduction for visualizing single-cell data using UMAP

Today's lecture

- 1 Single-cell sequencing technology
- 2 Basic quality control
- 3 Additional Q/C tools
- 4 Doublet detection in single-cell data**
- 5 Data normalization across many batches

What if we capture more than one cell in a droplet?



Macosko *et al.*, *Cell* (2015)

What is a doublet in single-cell data?

Biological/technical definition:

- One or more cells captured (usually at most two cells by chance)
- Thus, multiple cells accidental share the same cell barcode sequence
- Not so clear in general... since we missed the chance to assign different tags to different cells encapsulated in the same droplet.

Statistical definition:

- If we could find marker genes of multiple cell types are simultaneously expressed...
- An unvetted approach: Find ambiguous/intermediate coordinates in PCA/tSNE/UMAP (after removing ambient cells).

Can we create artificial doublets?

A straightforward definition (used in DoubletFinder):

For each cell i :

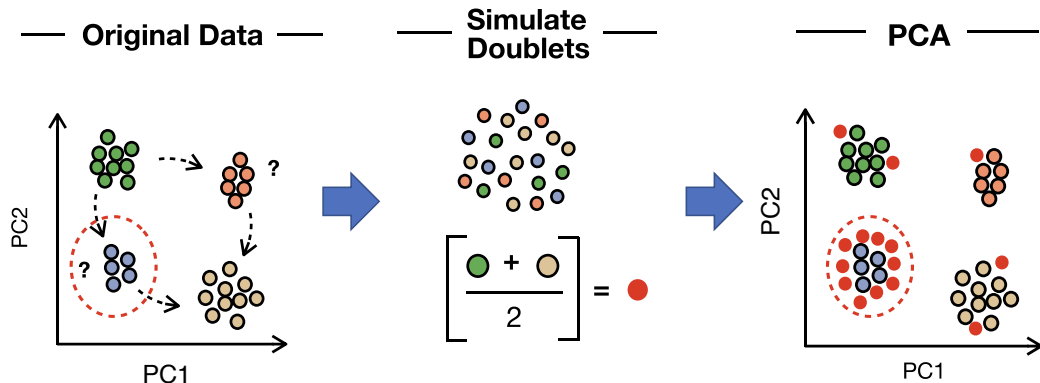
- Take some other j by random selection
- Create an artificial doublet

$$\tilde{\mathbf{x}} \leftarrow \frac{1}{2}(\mathbf{x}_i + \mathbf{x}_j)$$

Some thought questions:

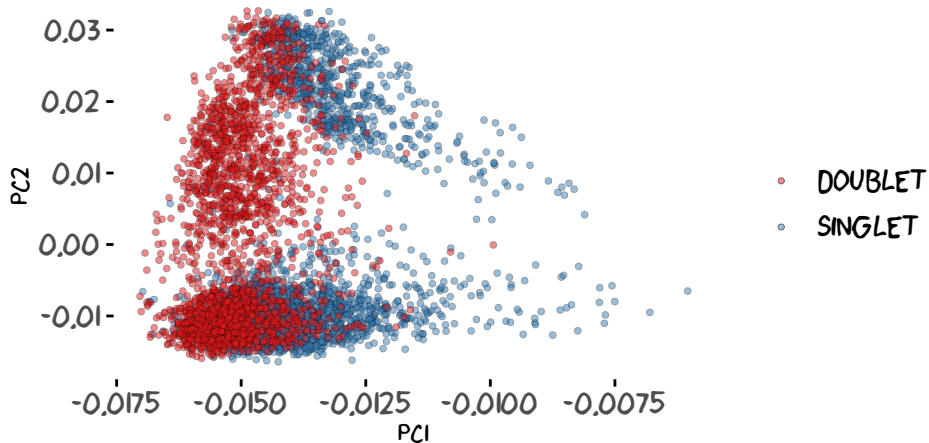
- Doublets within the same cell type?
- Doublets between the different cell types?

k-Nearest Neighbour classification for doublet detection

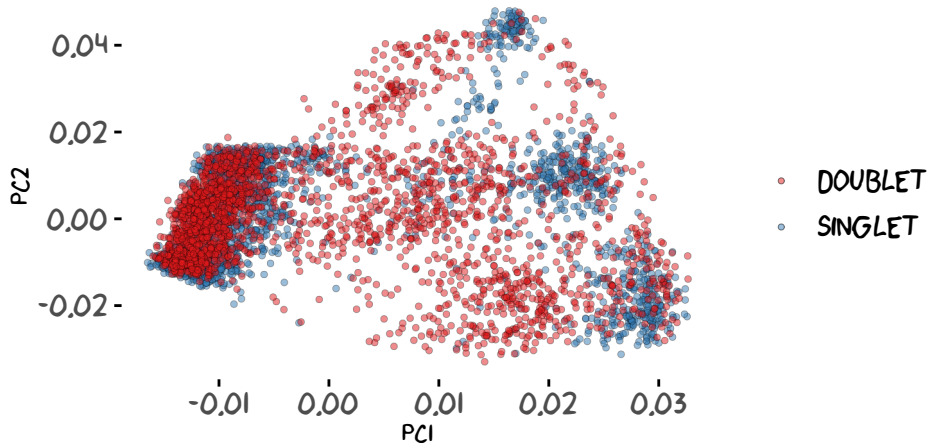


McGinnis et al. Cell Systems (2019)

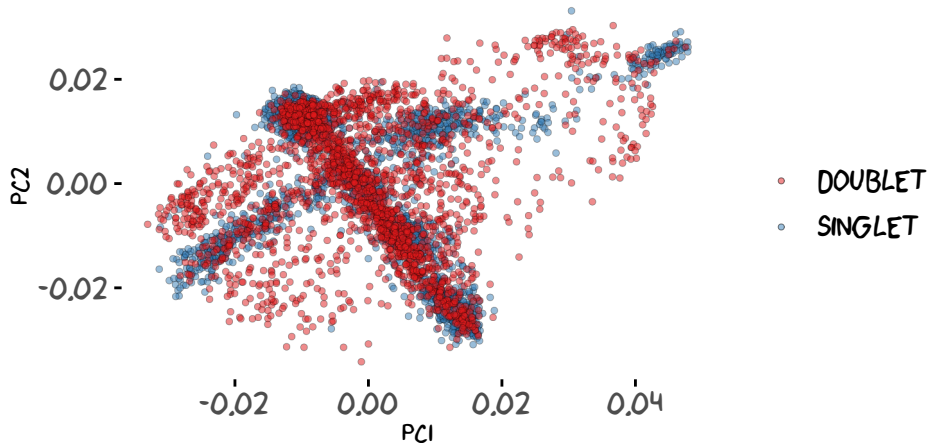
Can you tell the difference by a quick visual inspection?



Can you tell the difference by a quick visual inspection?



Can you tell the difference by a quick visual inspection?



Can we design a classifier to distinguish singlets vs. doublets?

k-Nearest Neighbour classification for doublet detection

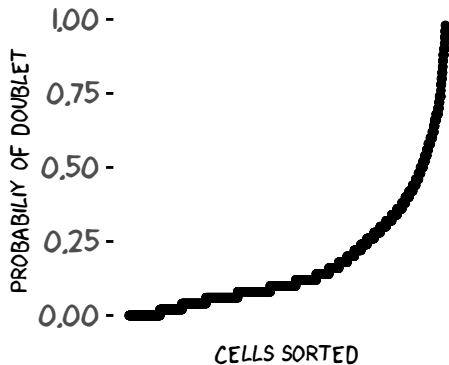
- Step 1. Create artificial doublets, $\tilde{x}$
- Step 2. Mix them with the original cells and perform PCA
- Step 3. Find nearest neighbours of the original cells (using $\#PC=10$)
- Step 4. Count the number of doublets in the neighbourhood

k-Nearest Neighbour classification of doublets

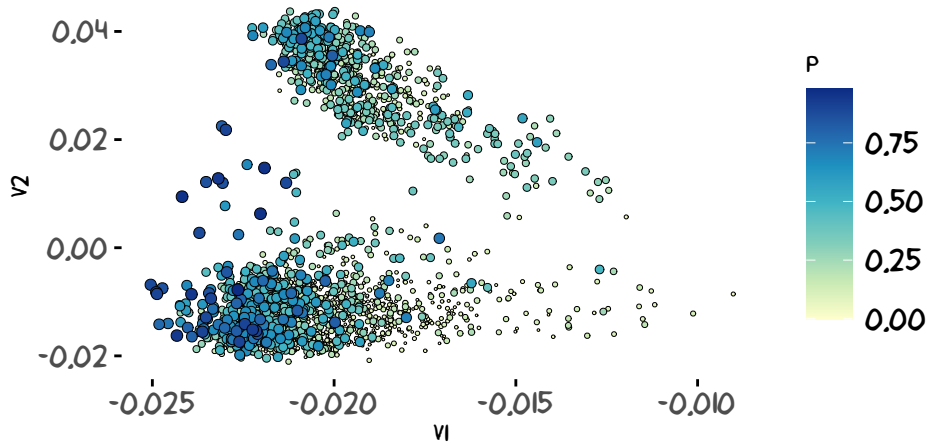
- Q: How many of my neighbours are indeed a doublet?

$$\hat{P}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} I\{j \text{ is a doublet}\}$$

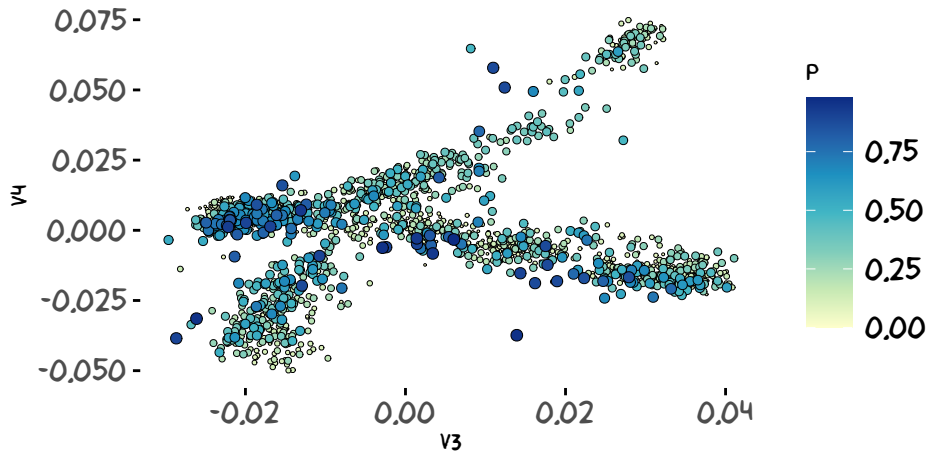
Key assumption: There is a principal component that can set apart hidden doublets from the most of singlets.



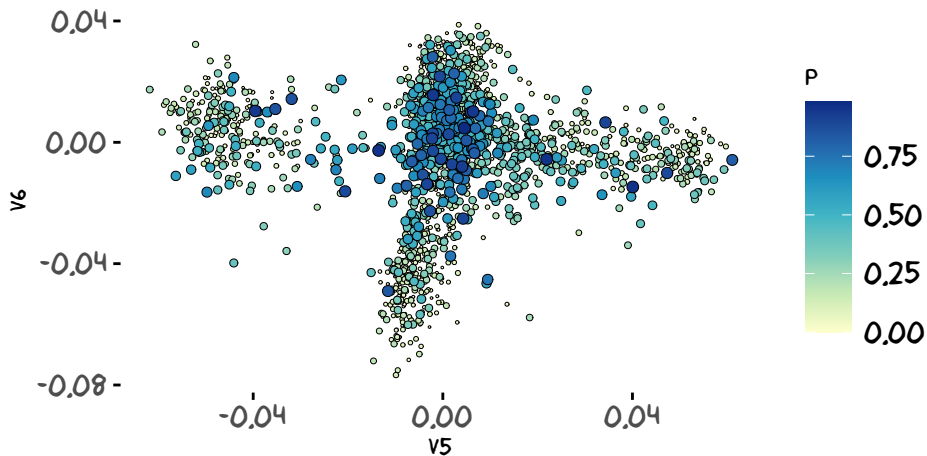
Doublets enriched in the outskirts of clusters



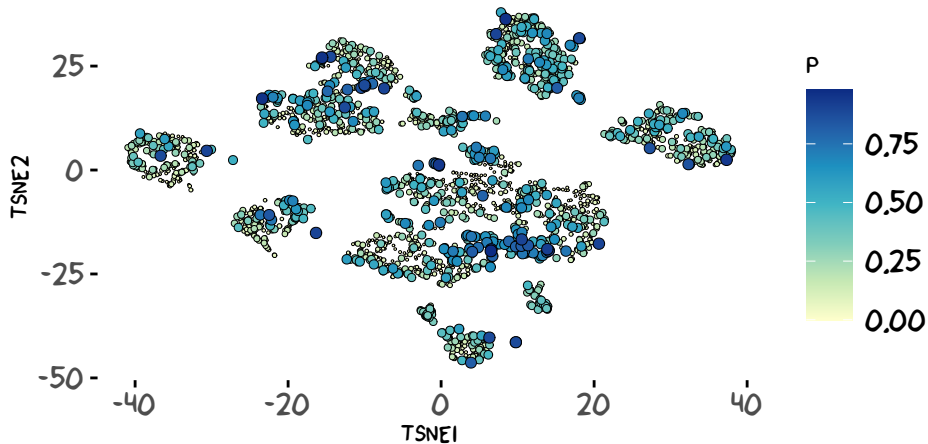
Doublets enriched in the outskirts of clusters



Doublets enriched in the outskirts of clusters



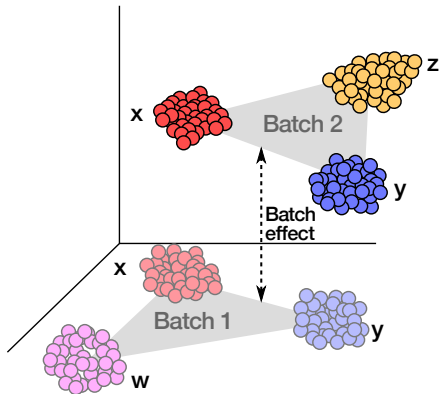
Doublets enriched in the outskirts of clusters



Today's lecture

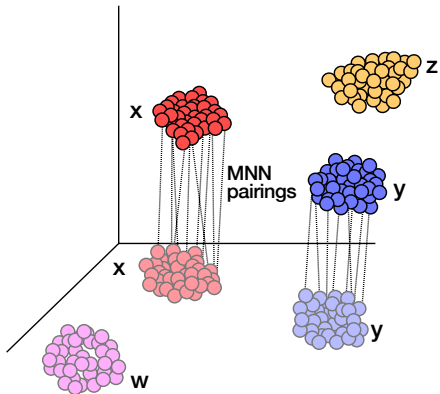
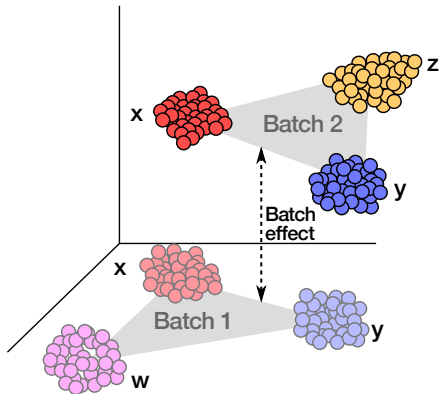
- 1 Single-cell sequencing technology
- 2 Basic quality control
- 3 Additional Q/C tools
- 4 Doublet detection in single-cell data
- 5 Data normalization across many batches**

Batch normalization aims to minimize the difference between nearest cells across different batches



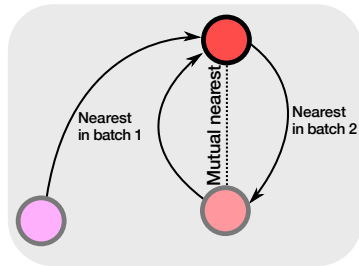
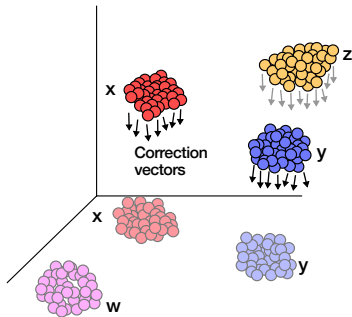
Haghverdi, ..., and Marioni, *Nature Biotechnology* (2018)

Batch normalization aims to minimize the difference between nearest cells across different batches



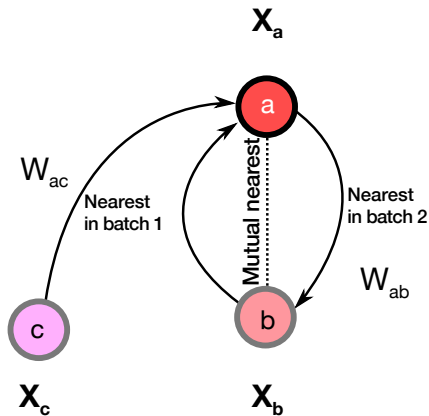
Haghverdi, ..., and Marioni, *Nature Biotechnology* (2018)

Batch normalization aims to minimize the difference between nearest cells across different batches



Haghverdi, ..., and Marioni, *Nature Biotechnology* (2018)

Batch normalization aims to minimize the difference between nearest cells across different batches



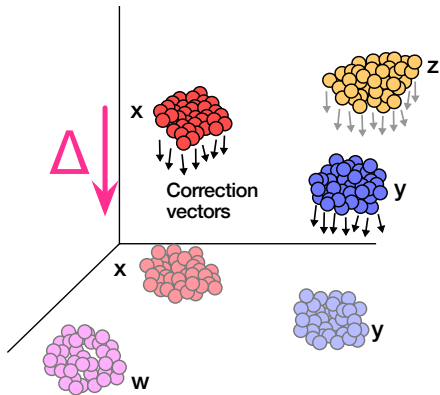
What is the gap Δ between the batches?

$$\min_{\Delta} \sum_{a,b} W_{ab} \|\mathbf{x}_a - \mathbf{x}_b - \Delta\|_2$$

A key assumption:

$$0 \approx W_{ac} < W_{ab}$$

Batch normalization aims to minimize the difference between nearest cells across different batches



What is the gap Δ between the batches?

$$\min_{\Delta} \sum_{a,b} W_{ab} \|\mathbf{x}_a - \mathbf{x}_b - \Delta\|_2$$

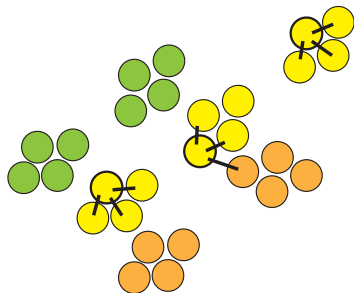
Fixed point (local) optimal solution:

$$\Delta \leftarrow \frac{\sum_{a,b} W_{ab} (\mathbf{x}_a - \mathbf{x}_b)}{\sum_b W_{ab}}$$

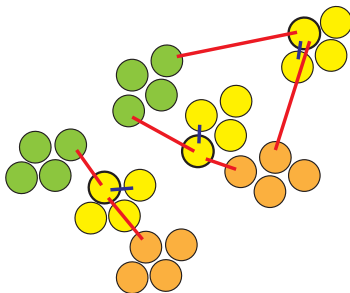
A batch-balancing k-nearest neighbour graph

BBKNN method strikes balance between over- and under-normalization

K-Nearest Neighbour



Batch Balanced K-Nearest Neighbour

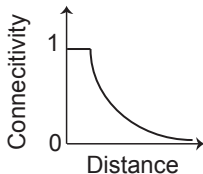
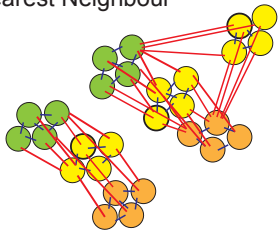


What kind of differences in due to inter-batch, technical discrepancy, not inter-cell-type divergence?

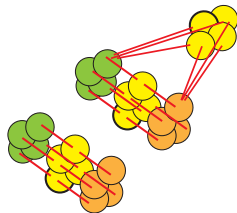
A batch-balancing k-nearest neighbour graph

BBKNN method strikes balance between over- and under-normalization

Batch Balanced
K-Nearest Neighbour



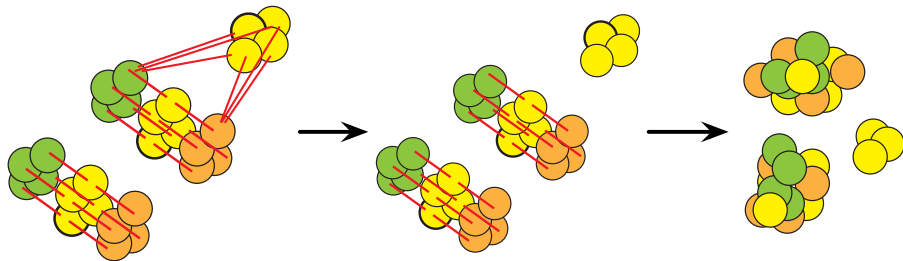
Converting distances
to connectivities
(see Methods)



Polanski, ..., Teichmann *Bioinformatics* (2019)

A batch-balancing k-nearest neighbour graph

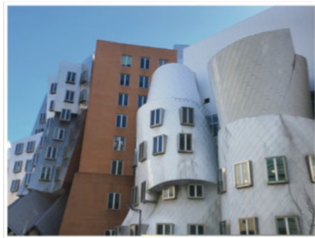
BBKNN method strikes balance between over- and under-normalization



Polanski, ..., Teichmann *Bioinformatics* (2019)

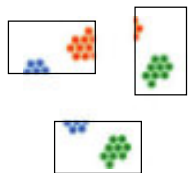


snapshots
of many data

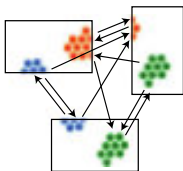


panorama stitched
together

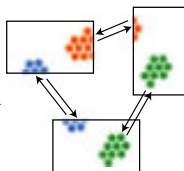
How do we
integrate multiple
samples/batches?



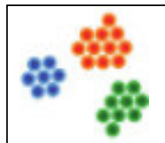
Collect many
single-cell RNA-seq
experiments



Find nearest
neighbours
across data sets



Keep mutually
neighbouring
cell pairs

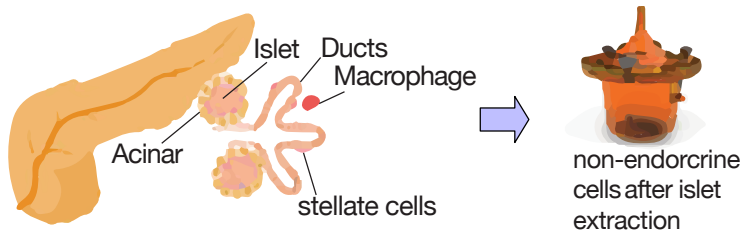


Create
single-cell
panorama

Scanorama:
mutual nearest
neighbourhood-
based data
integration

Example: pancreatic cells across three batches

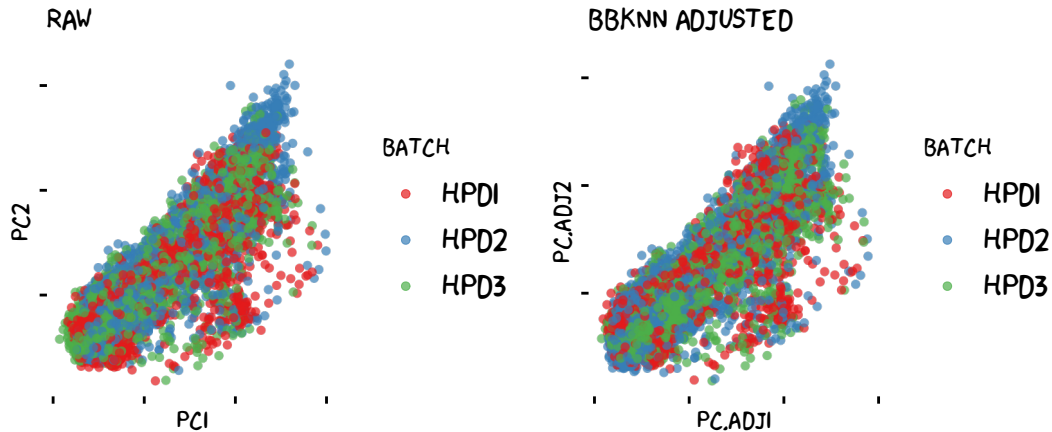
Single-cell RNA-seq data from three donors (three batches)



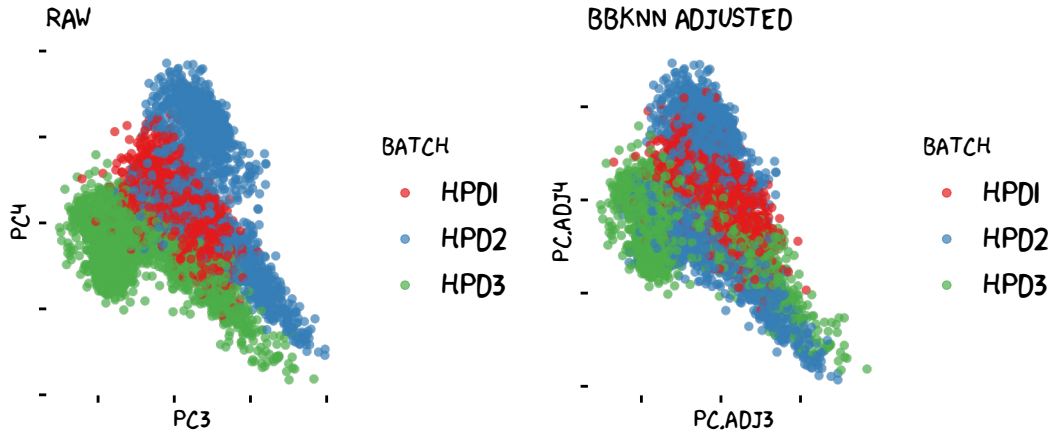
Goal: Remove potential batch effects across different donors. (1) Construct BBKNN graphs between cells; (2) compute average discrepancy Δ between batches in the PC space; (3) adjust them.

Qadir, *et al.*, *PNAS* (2020)

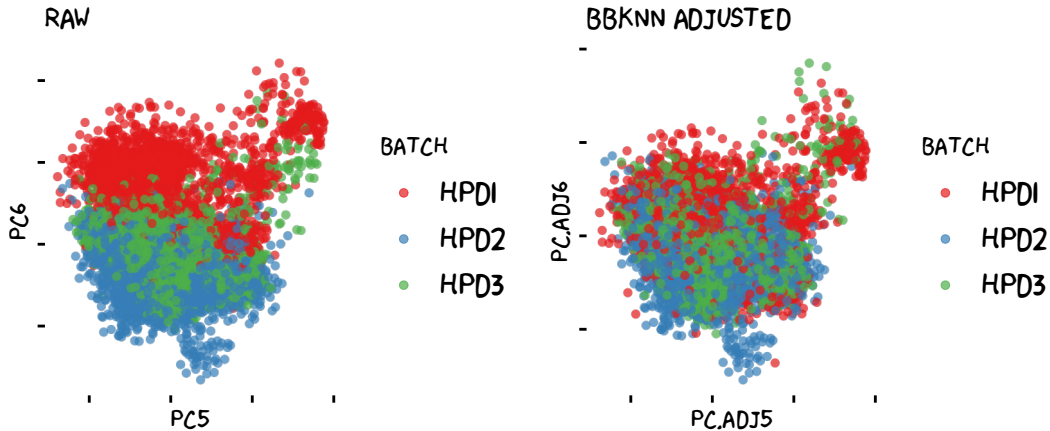
BBKNN-guided normalization



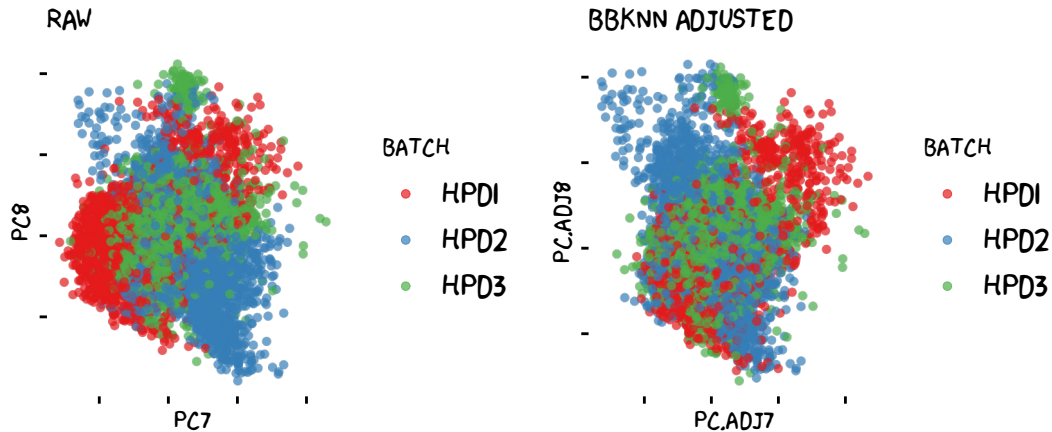
BBKNN-guided normalization



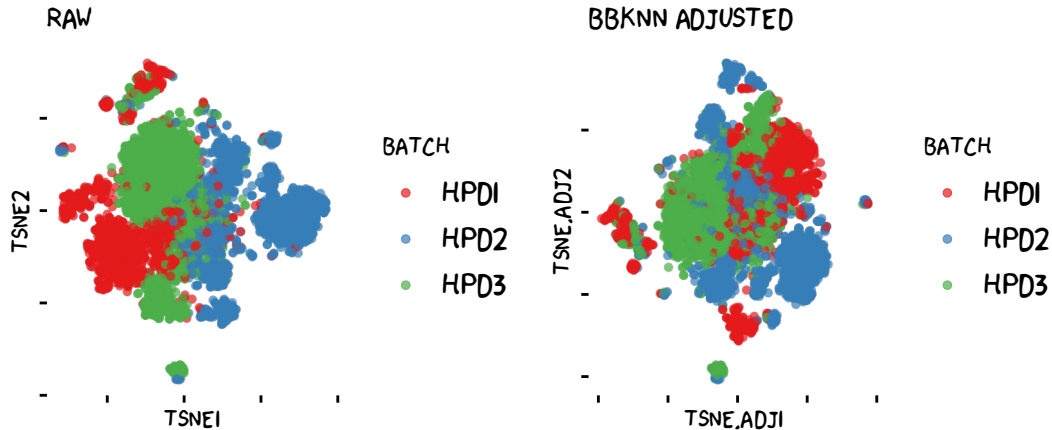
BBKNN-guided normalization



BBKNN-guided normalization



BBKNN-guided normalization



Discussions

- What can we do with BBKNN graphs?
- Why do we need batch normalization?
- Is it possible to over-correct the differences?
- Is it also possible to under-correct the differences?